# Distributed Inference Serving - vLLM, LMCache, NIXL and llm-d

2025/06/25

Mikiya Michishita(@aztecher)

# Disclaimer

- 本資料の内容は、2025年06月25日時点の情報や実装に基づいています。
  - 実装についてもすべてのプログラムが最新ブランチを参照しているわけではなく、あくまでllm-dの当時の最新ブランチが依存としているバージョンを参照しています。
- 将来にわたり情報等が更新される可能性があります。
- 著者もキャッチアップしながら学んでいる身のため不正確な情報がある可能性があります
  - もし間違いを見つけたら報告いただけると幸いです。
- 本資料でターゲットにする話題はLLMを分散環境でサービングするケースです
  - 基本的には小規模なモデルやLLMではないモデルについては取り扱いません

# Distributed LLM Inference Serving

- LLMの推論はその処理の特性、応答特性や、ユーザトラフィックの変化などにより、分散環境で実行することでメリットが生まれるケースがある
  - 当然それ以外にも単順にモデルに対する要求が急激に増加している背景もある
- 本資料ではこれらの特性や変化を元にしつつ、vLLM / LMCache / NIXL / llm-dなどのソフトウェアを取り上げ、アプリケーションレイヤーの動作や最適化、分散推論インフラについて理解を深めていく

# LLMへのリクエストとLLMの処理の特性

LLMへのリクエストやその処理には非常に重要な特性がある

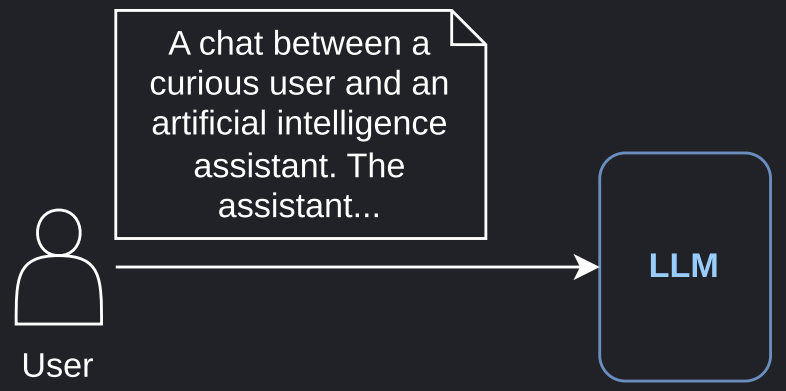
- ユーザリクエストの多様化と複雑化
  - ユーザが入力するPromptは多様であり複雑、入力長もリクエスト毎に大きく異なる
  - 一方で、多くのリクエストは以前の会話の焼き回しを含む、という特性もある
- リクエストへの応答には異なる計算フェーズが存在し、それぞれ処理特性が異なる

これらの特性について理解し、最適化するための手法を知ることが、LLM推論向け基盤を理解する第一歩になる

# ユーザリクエストの変化

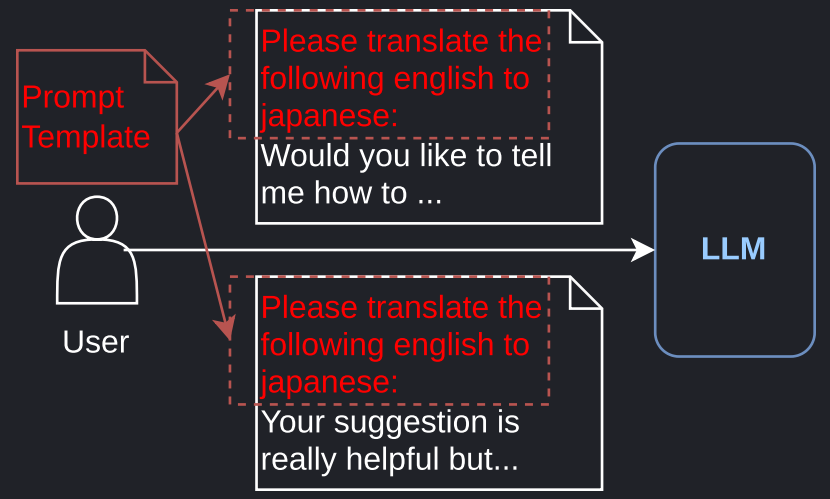
LLMへのリクエストは従来のウェブアプリケーションにおける一般的なユーザリクエストとは大きく異なる特性を持つ

- **従来**: シンプルかつそこまで多様ではない内容でデータ長も短い
- **LLMへのリクエスト**: リクエストは非常に複雑で多様でありデータ長は長い



# ユーザリクエストの特性とLLMによる計算の特性

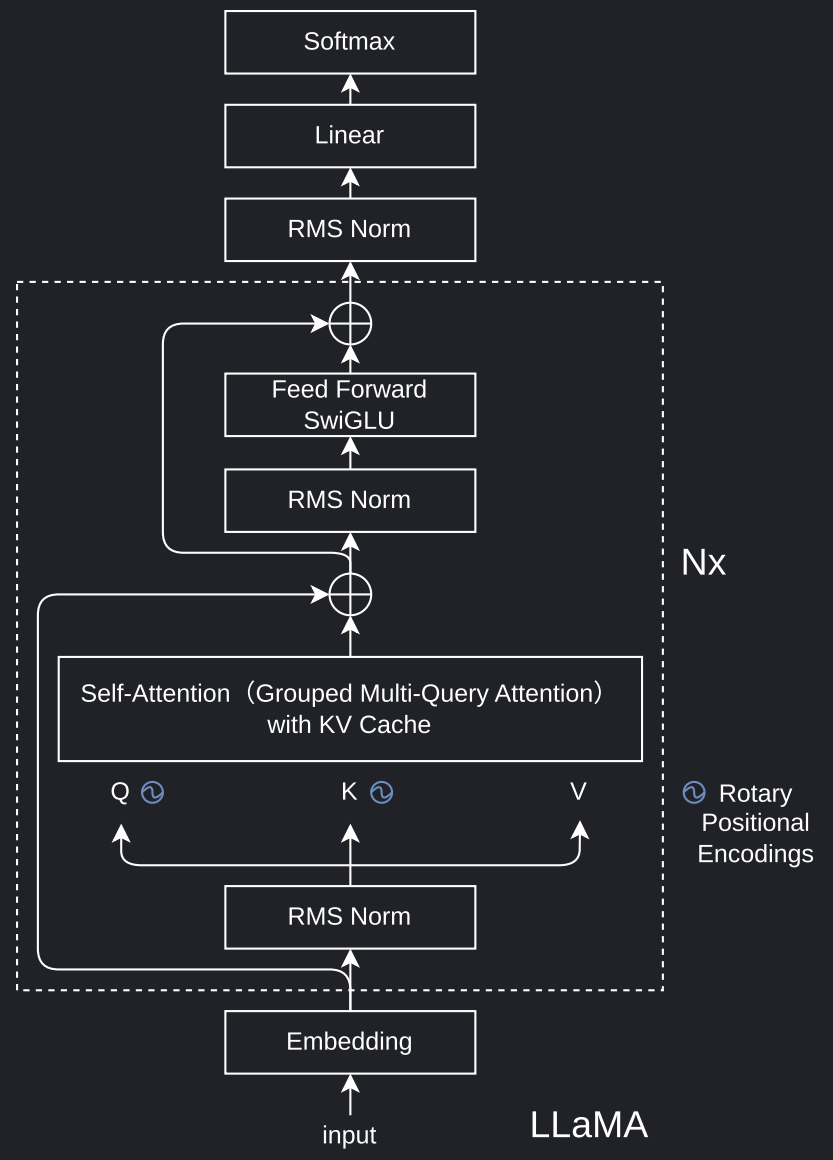
- リクエスト内容は従来に比べ多様化、複雑化する一方で、Prompt EngineeringやRAGなどのシステム化によって、プロンプト前半にテンプレート的な定型文が多用される。
- LLMでは入力トークンに対して計算処理を行うが、文章が定形である場合は計算結果が同じになる特性がある。



# Transformer Model

文章生成などの生成タスクに採用される多くのモデルは、Decoder-onlyのTransformerモデル（Autoregressive Model）である。

Autoregressive Modelでは、モデルが生成したトークンを系列の末尾に加え、再度次のトークンの予測計算を行うような機構になっている。



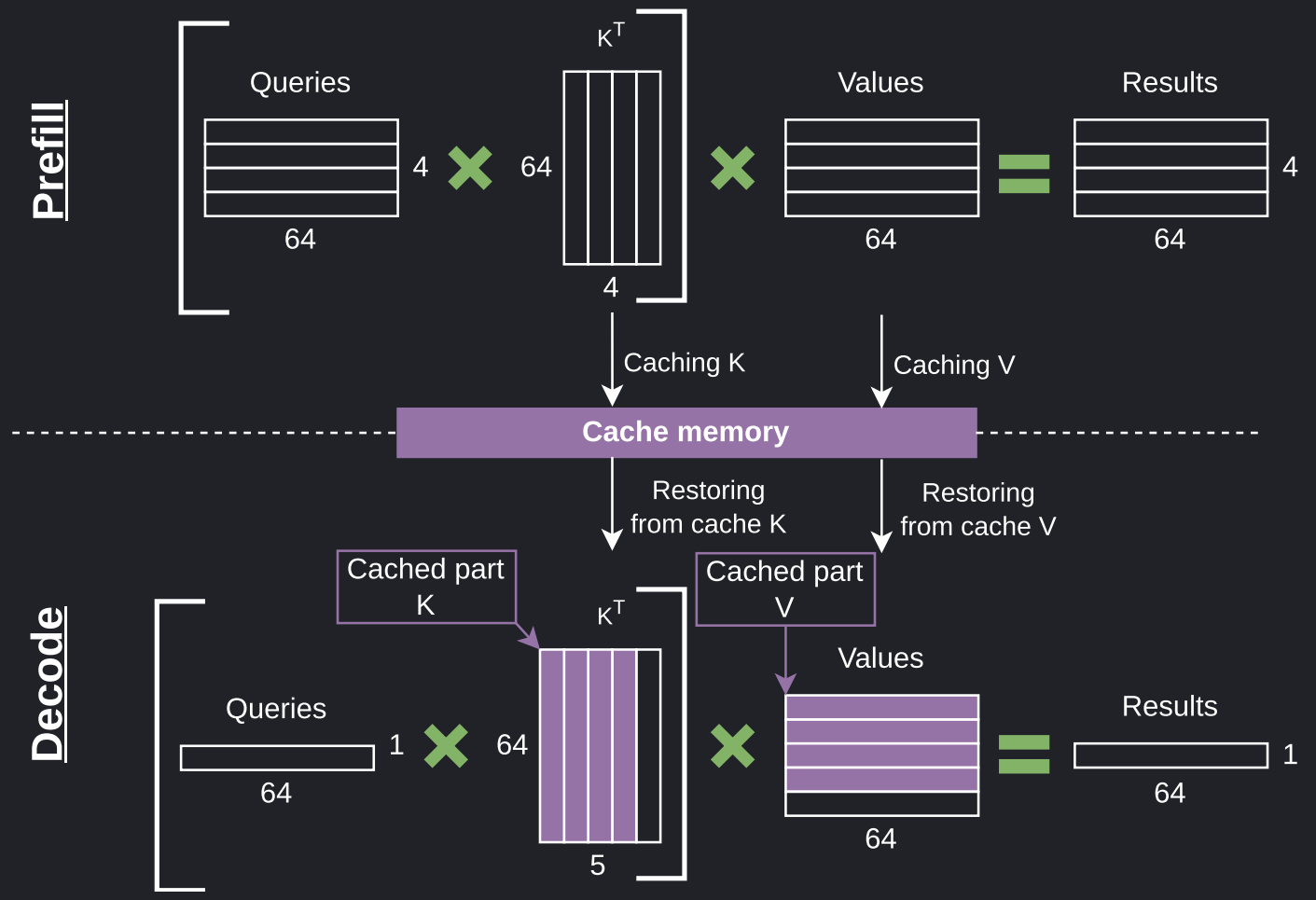
# Attention

モデルの構造中の重要な処理として、Attentionの計算がある。
ただし、Autoregressive Modelの場合はこのAttentionの計算は新しいトークンを除いてただの再計算になってしまい、特にこれは入力トークン長に依存して計算量も大きくなる

そのため、計算結果をキャッシュし、以降の計算で使いまわすことで計算量を減らそう、という発想になる。ここで出てくる「計算結果のキャッシュ」が KV-cache である

*TransformerやAttentionの話はたくさんの情報が検索すると出てくるのでここでは省略します。スライド最後のReferencesに自分が主に参照した資料については掲載しています

# KV-cache overview




ref: Mastering LLM Techniques: Inference Optimization

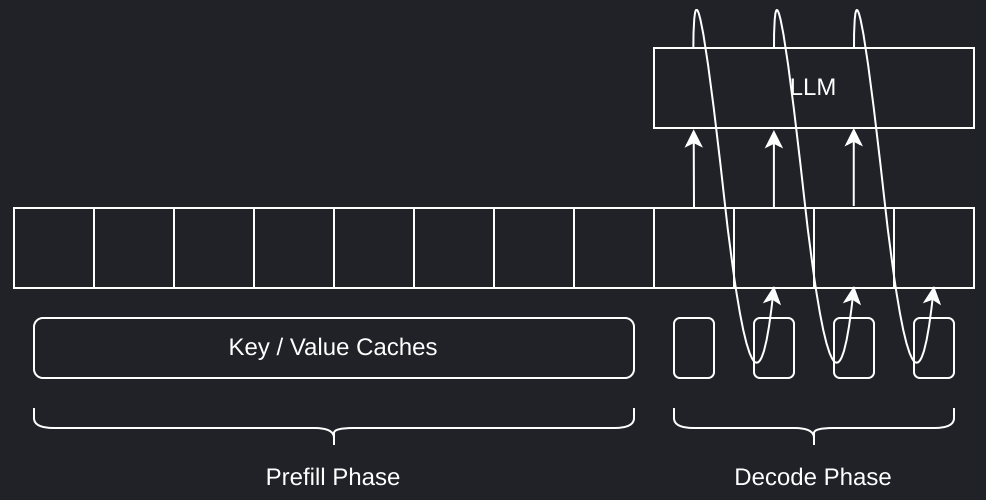
# Prefill phase and Decode phase

### Prefill phase

- 入力トークンを処理し、最初のトークンを生成
- 入力トークンに対応するKV-cacheも生成
- 行列演算が主となりCompute Boundな処理

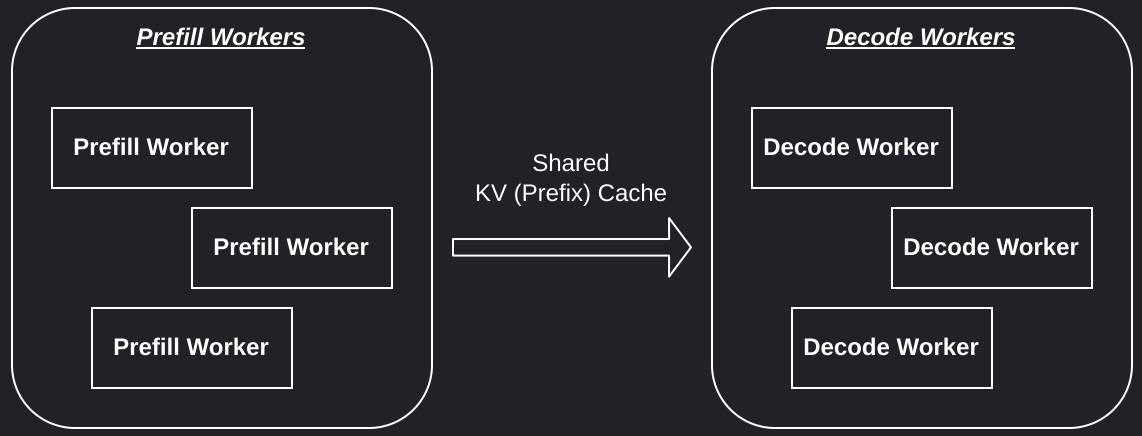
### Decode phase

- 直前のtokenをベースに後続の出力を生成する
- KV-cacheによる計算結果の再利用ができ計算処理は重くないが、主記憶からGPUへのデータ転送に時間がかかるMemory Boundな処理



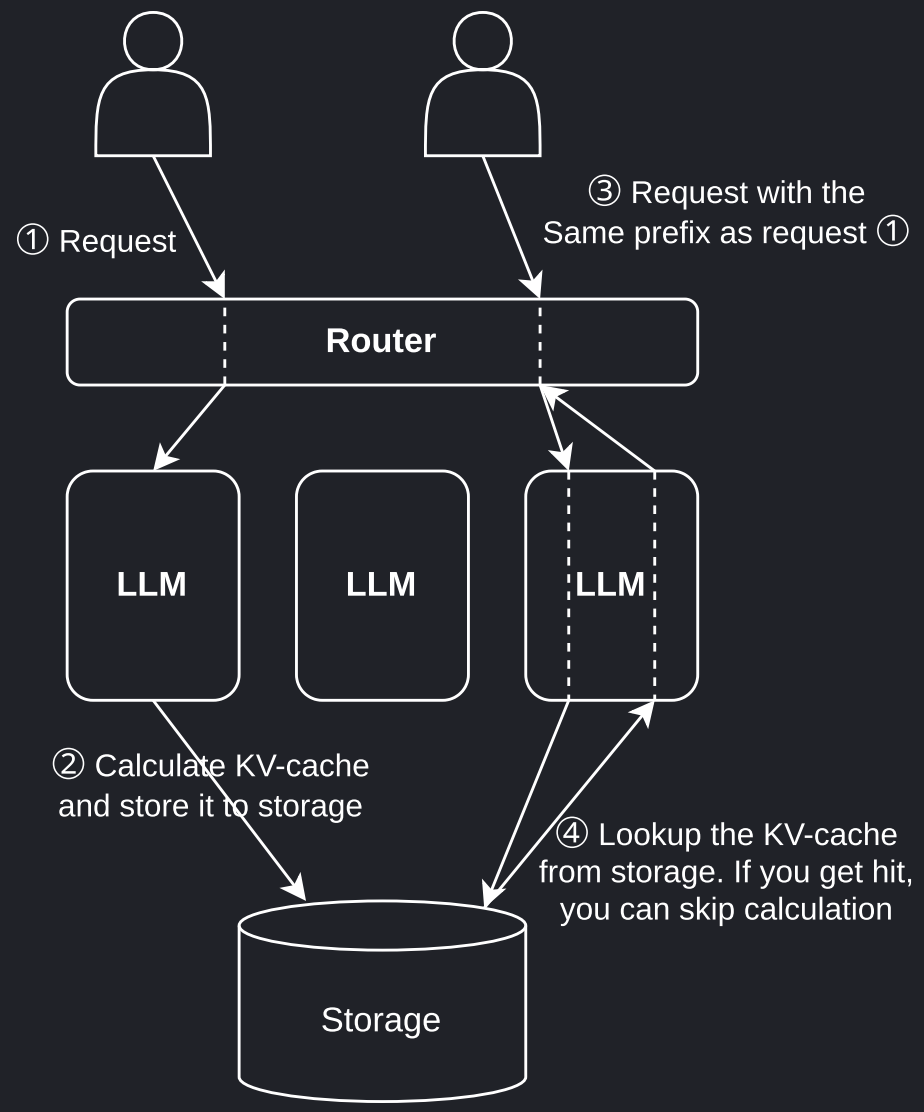
# Prefill-Decode (PD) Disaggregation

- Prefill処理とDecode処理を別のWorkerに分離
- 異なるボトルネックを適切に分離し効率的に処理しそれぞれの処理効率を上げる手法
- 一方で「**KV-cacheをいかに共有するか**」,「**どのようにリクエストをルーティングするか**」など新たな考慮事項が出現することになる。



# KV-cache sharing

- 同一モデルであれば、すでにKV-cacheを作成済みのトークンは使いまわすことができる。
- KV-cacheを外部ストレージなどに格納し、LLMサーバ間で共有することで、Prefillの計算処理の一部をスキップすることができ高速化が見込まれる
- 復数サーバで処理することで、より多くのユーザアクセスをより受け付けられるようになり、ノード障害に対する可用性等の面でも有利な面が出てくる。



# vLLM

# vLLMとは

- vLLMは元はUC BerkeleyのSky Computing Labによって開発されたプロジェクト
- 高速で簡単に利用できるLLM inference & serving engine
- Community-driven project
- 様々なモデルを様々なリソース（NVIDIA GPU, AMD GPU, Google TPU, ...etc）で動作させることができる
- 近年急成長しているソフトウェア（主観）で、様々な最新技術を取り込んでいる先進的な推論フレームワーク（主観）

正しいインフラは正しいアプリケーションの理解の上で成り立つので、本節ではvLLMをベースにLLM推論の技術について理解していく

# vLLM core technologies

- P/D disaggregation (これまで議論してきた通り)
- Paged Attention
- Automatic Prefix-Cache
- Continuous Batching
- Speculative Decoding (本資料ではskip)
- Chunked Prefill（本資料ではskip）
- Quantilization (本資料ではskip)

# Paged Attention (1/6)

- 従来の手法では、KV-cacheに利用するGPUメモリは、モデルが扱える**最大トークン数分を連続した領域に事前に確保**するというナイーブな実装になっていた。
- これは様々な課題を抱えていた
  - リクエストに対して余分にメモリを確保するケースが多々発生しメモリ効率が悪い
  - KV-cacheをリクエスト間で共有できない（リクエスト毎に上記の動きをするため）
  - ...etc

より効率的なKV-cacheのメモリ管理手法として、PagedAttentionが提案されvLLMに実装された。

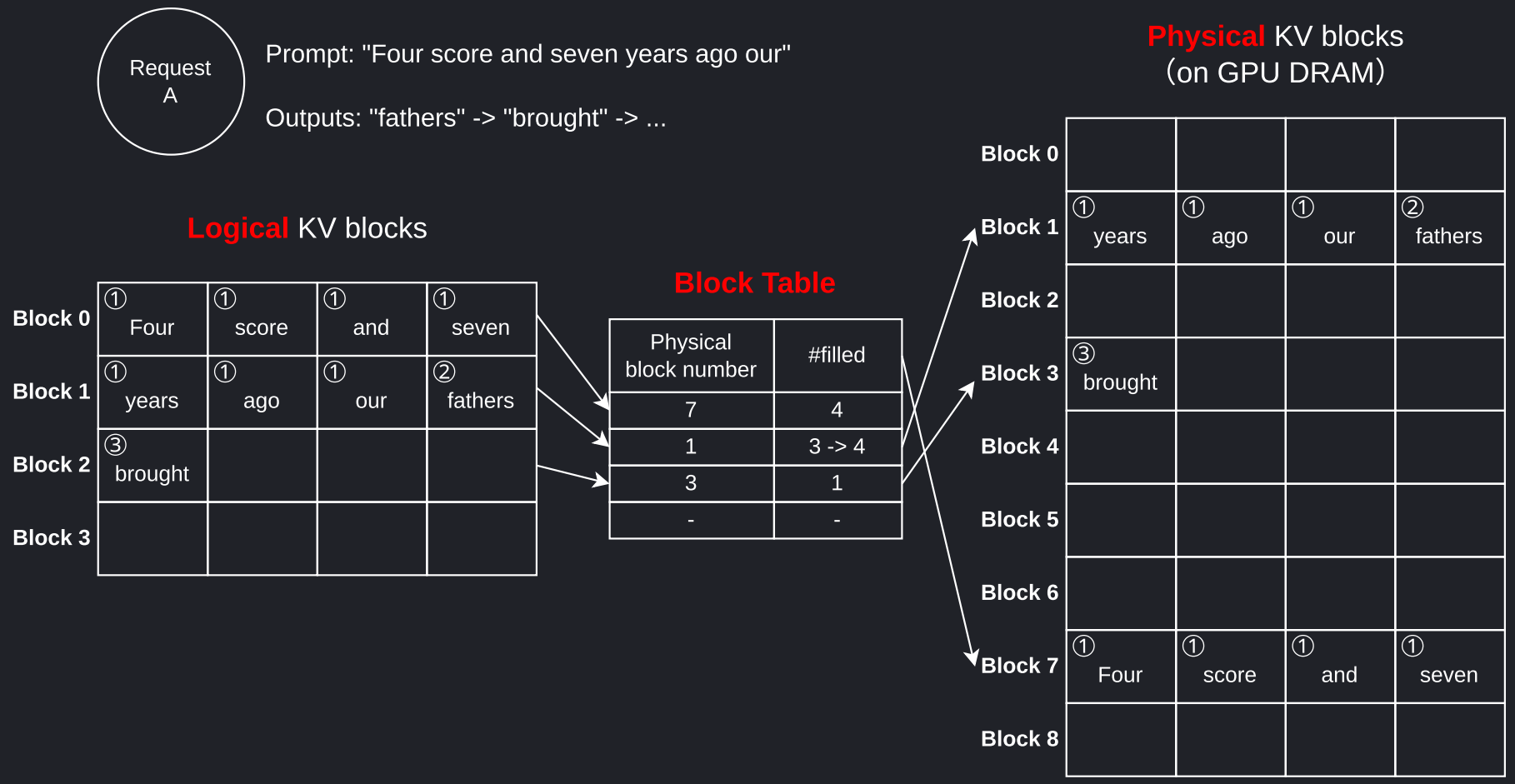
# Paged Attention (2/6)

PagedAttention: OSのページングを参考に考案されたKV-cache管理手法

- KV-cacheをKV blocksに分割する
- KV blocksは固定サイズのメモリチャンクで、Logical KV blocksとPhysical KV blocksの2つのブロックが定義される
- Logical KV blocksではトークンが自然な順序で格納され、Physical KV blocksでは連続したブロックの任意の場所にトークンが配置される
- Logical KV blocksとPhysical KV blocksのマッピングは**Block Table**と呼ばれる新しいデータ構造がに保存される

これまでの静的な固定長のメモリ割り当てから動的な割り当てになり、かつフラグメンテーションの問題を解決できる手法になっている。

# Paged Attention (3/6)




ref: Efficient Memory Management for Large Language Model Serving with Paged Attention

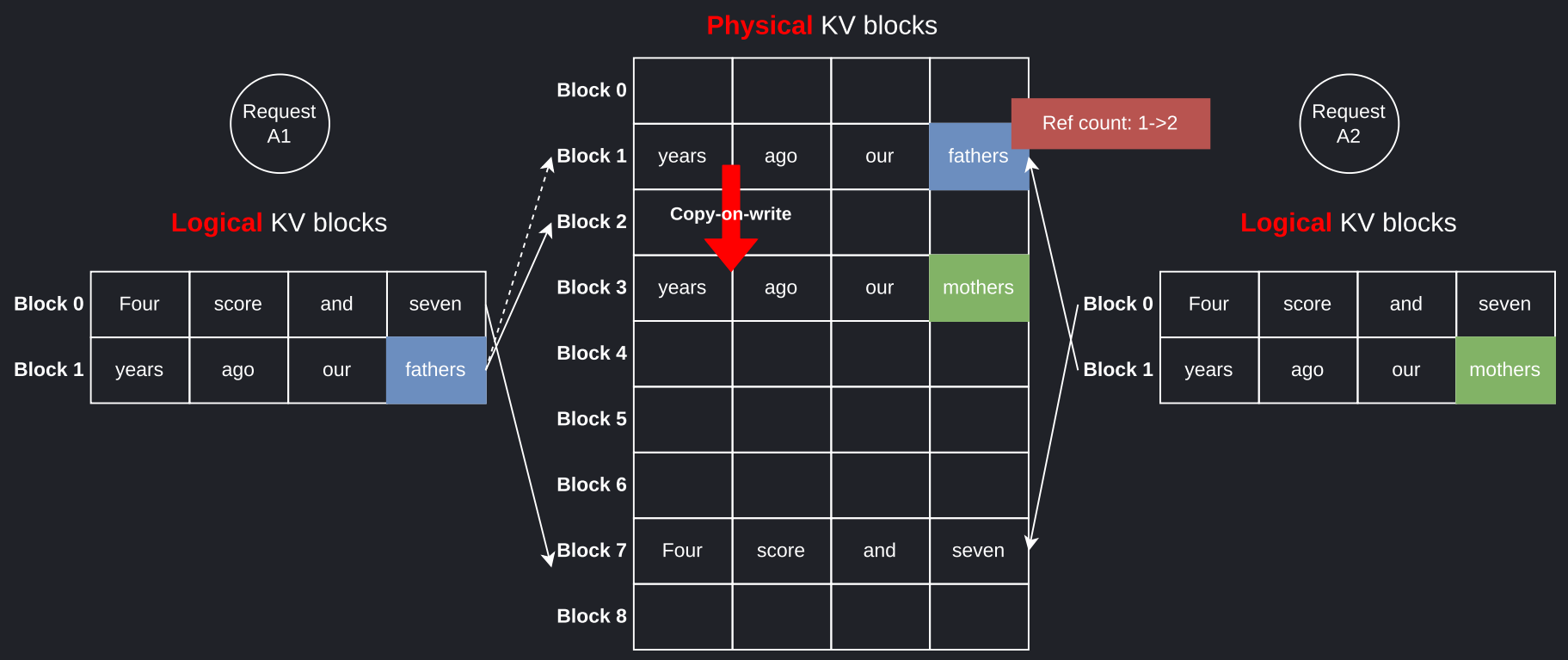
# Paged Attention (4/6)

① 入力プロンプトの7つのトークンをLogical KV blocks（Block 0, 1）とPhysical KV blocks(Block 7, 1)にマッピングする。Attentionアルゴリズムによって得られたKV-cacheはLogical KVblockのBlock 0, 1に格納する
② 新しく生成したトークン（fathers）は、Logical KV blocksで空いているBlock 1のスロットに割り当てられる。新しく生成したKV-cacheもそこに保存され、Block tableの#filledレコードが更新される。
③ 次の生成トークン（brought）はLogical KV blocksが満杯のため、新しいLogical KV block (Block 2)に保存する。このとき新しいPhysical KV block(Block 3)を紐付け、それをBlock Tableに記載する

# Paged Attention (5/6)

- 既存のシステムでは、シーケンス毎のKV-cacheが連続した別々のスペースに格納されるため、メモリ共有ができなかった。
- Paged Attentionにより、シーケンス間でメモリを共有することができるようになる。
  - 実際は安全な共有を保証するために、物理ブロックの参照カウント（Ref count）を追跡し、Copy-on-Writeメカニズムによって必要に応じてデータをコピーする。
- 同じリクエストに関連する異なるシーケンス間、あるいは異なるリクエスト間でもブロック単位でメモリの共有が可能になる。
  - 特に同一プロンプトから複数のシーケンスを生成する際の、並列サンプリングやビームサーチなどのサンプリングアルゴリズムにおいて、メモリオーバーヘッドを大幅に削減することができる。

# Paged Attention (6/6)




ref: Efficient Memory Management for Large Language Model Serving with Paged Attention

# vLLM core technologies

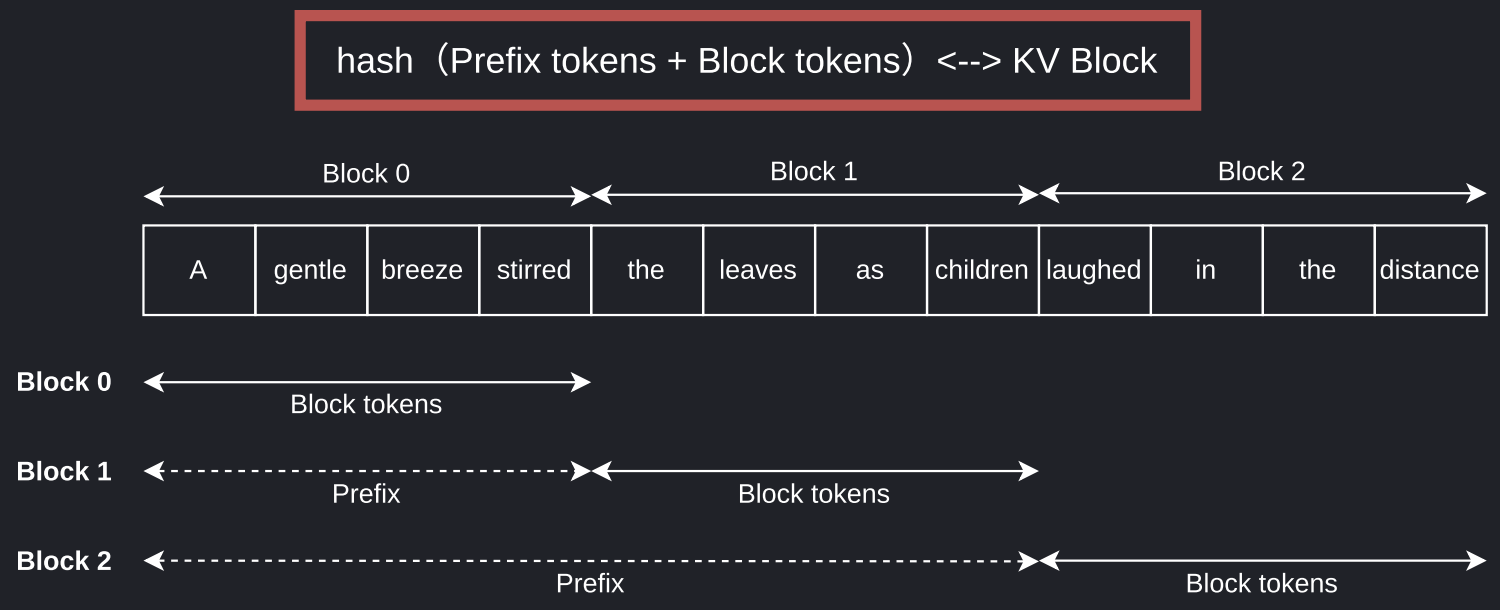
- P/D disaggregation (これまで議論してきた通り)
- Paged Attention
- Automatic Prefix-Cache
- Continuous Batching
- Speculative Decoding (本資料ではskip)
- Chunked Prefill（本資料ではskip）
- Quantilization (本資料ではskip)

# Automatic Prefix-Caching

- 前項で触れた「Request間でのKV-cacheを共有」を行うには、各Requestが参照可能なMappingの保持や管理、Lookup時のUnique Keyなどを考える必要がある。
- vLLMではこれをKV-cache Managerが担っている。
- Request間でのKV-cacheを共有は、**Prefillフェーズの計算効率を上げることに寄与する**

# Automatic Prefix-Caching

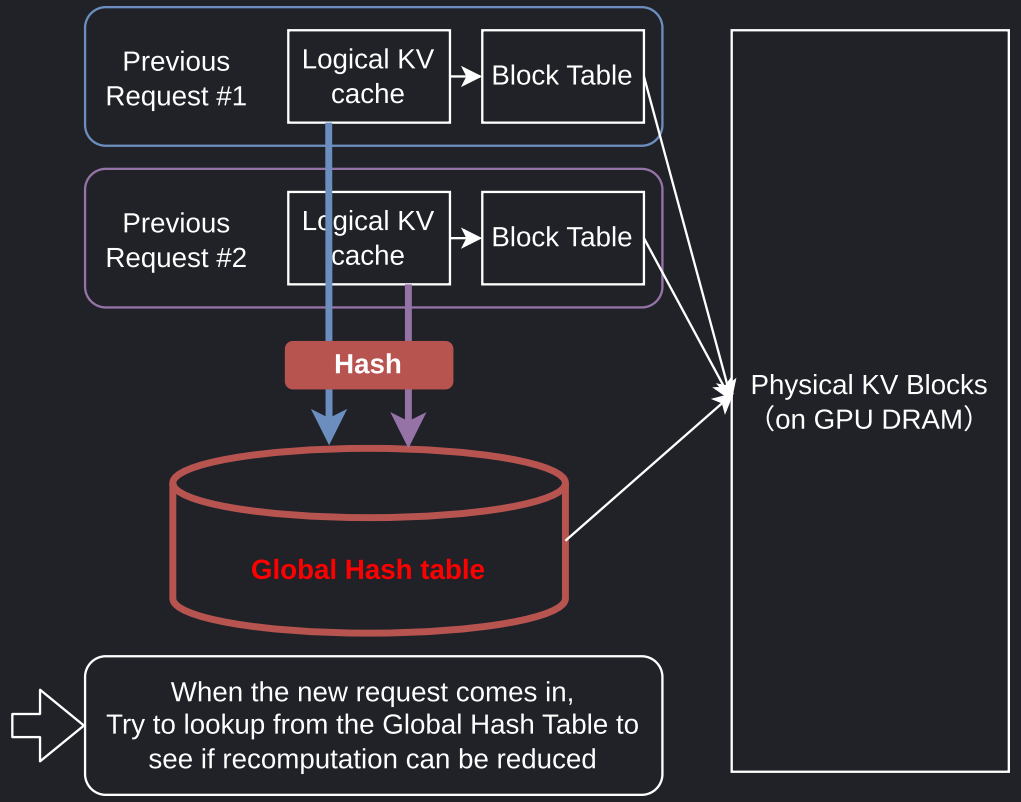
キャッシュをLookupするためのKeyを、Paged AttentionのBlockをベースとして計算する




ref: Automatic Prefix Caching

# Automatic Prefix-Caching

- Request毎に作成されるLogical KV cacheを元にhash値を計算し、Phyiscal KV Blocksとの関係をGlobalなHash Tableに格納する
- 新規Requestの入力トークンのうち、Prefixが一致する部分はGlobal Hash TableからLookup可能であり、再計算が不要になるため、Prefillフェーズの計算量削減につながる

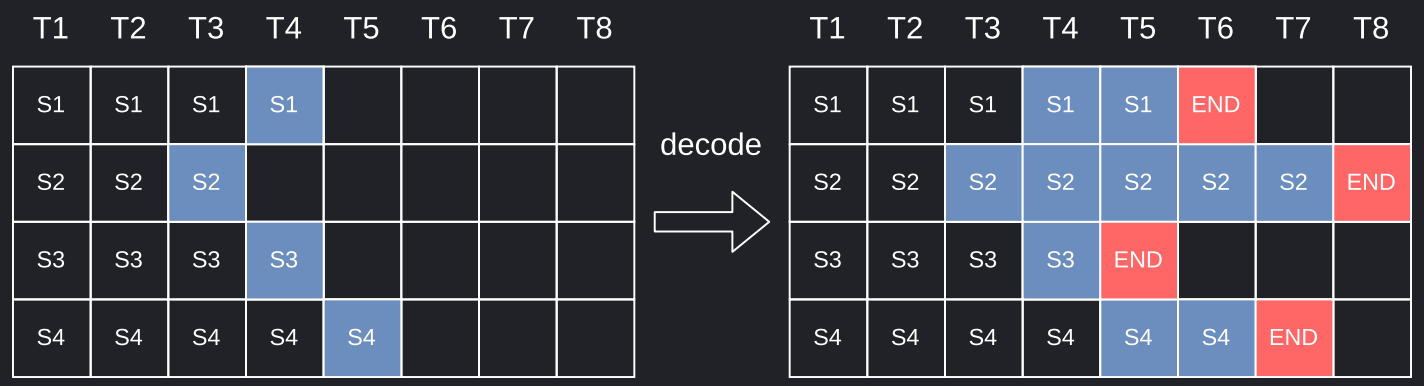


# vLLM core technologies

- P/D disaggregation (これまで議論してきた通り)
- Paged Attention
- Automatic Prefix-Cache
- Continuous Batching
- Speculative Decoding (本資料ではskip)
- Chunked Prefill（本資料ではskip）
- Quantilization (本資料ではskip)

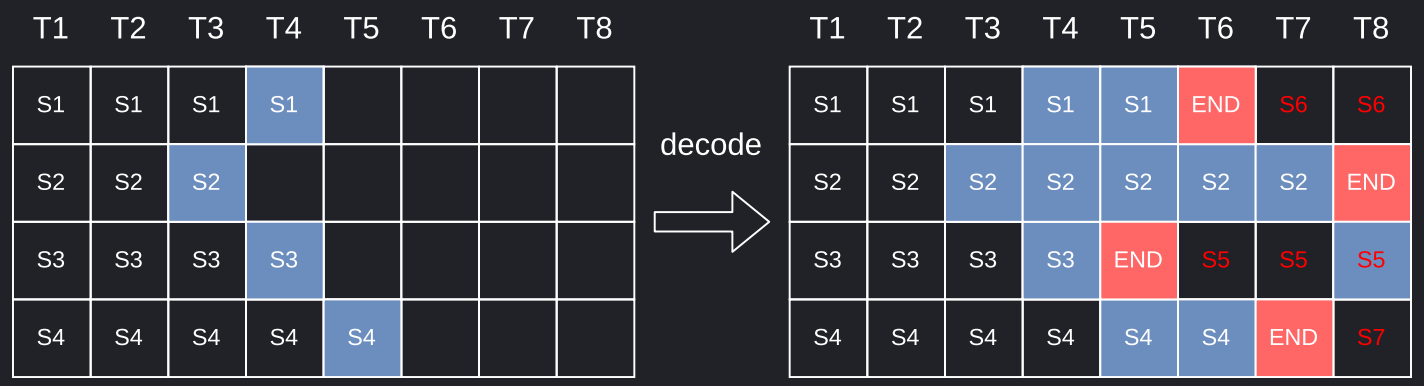
# Continuous Batching (1/2)

- LLMの推論リクエストに対しての、サーバサイドのバッチ処理の最適化を考える。
- 特徴的な点は、リクエストに対しての応答サイズ（出力トークン長さ）が不定であること
  - そもそもバッチで処理を行おうとすると、バッチの中で最も長い出力を生成する処理に引っ張られてしまう。



# Continuous Batching (2/2)

- LLM推論の新たなバッチ戦略としてContinuous Batchingが提案された*
  - バッチ中の生成を終了したシーケンスに対して、次のリクエストを詰めていくイメージ
  - iteration-level schedulingとも呼ばれる方式
- vLLMでもこのContinuous Batchingがサポートされている




*OSDI'22: Orca: A Distributed Serving System for Transformer-Based Generative Models

# vLLM KV Connector API, NIXL and LMCache

# How to share KV-cache?

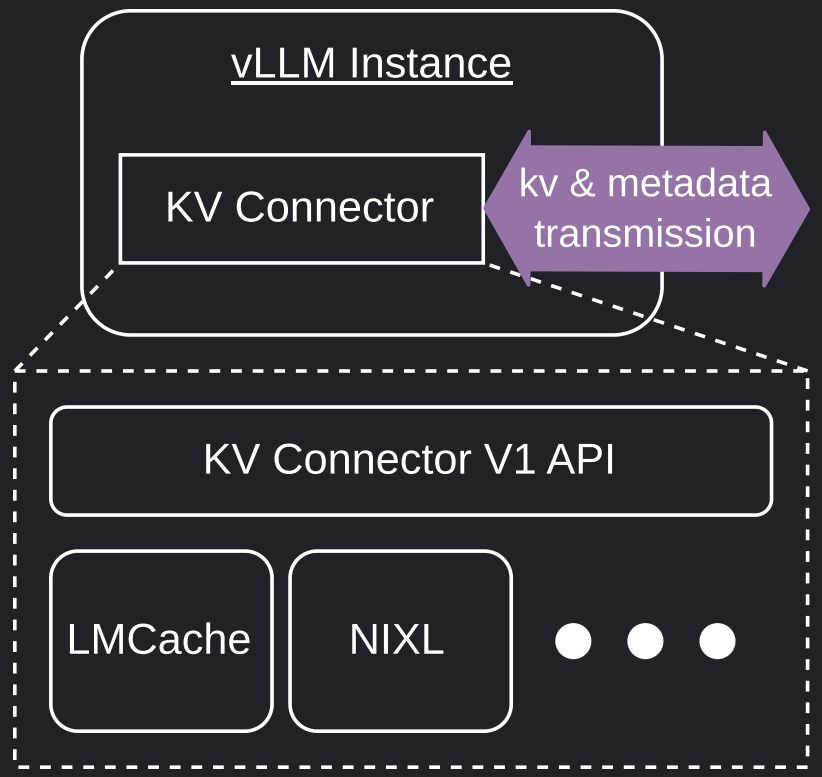
P/D Disaggregationを実現するためには、何かしらの方法によってKV-cacheを複数推論サーバ間で共有する必要がある。

- Peer-to-PeerでKV-cacheを送受信する
- 共有ストレージを利用してKV-cacheを共有する
- ...etc

vLLMではこの部分をコアロジックから分離しインターフェイスとすることでプラグイン可能にしている。

# KV Connector API

- **KV Connector API** はvLLMにおけるKV-cacheの管理を外部システムと連携するためのインターフェイス。
- 最近、V0から大幅にリファクタリングされたV1 APIが実装された。



# V1 Supported Connectors

現時点で確認できるV1実装としては以下がある

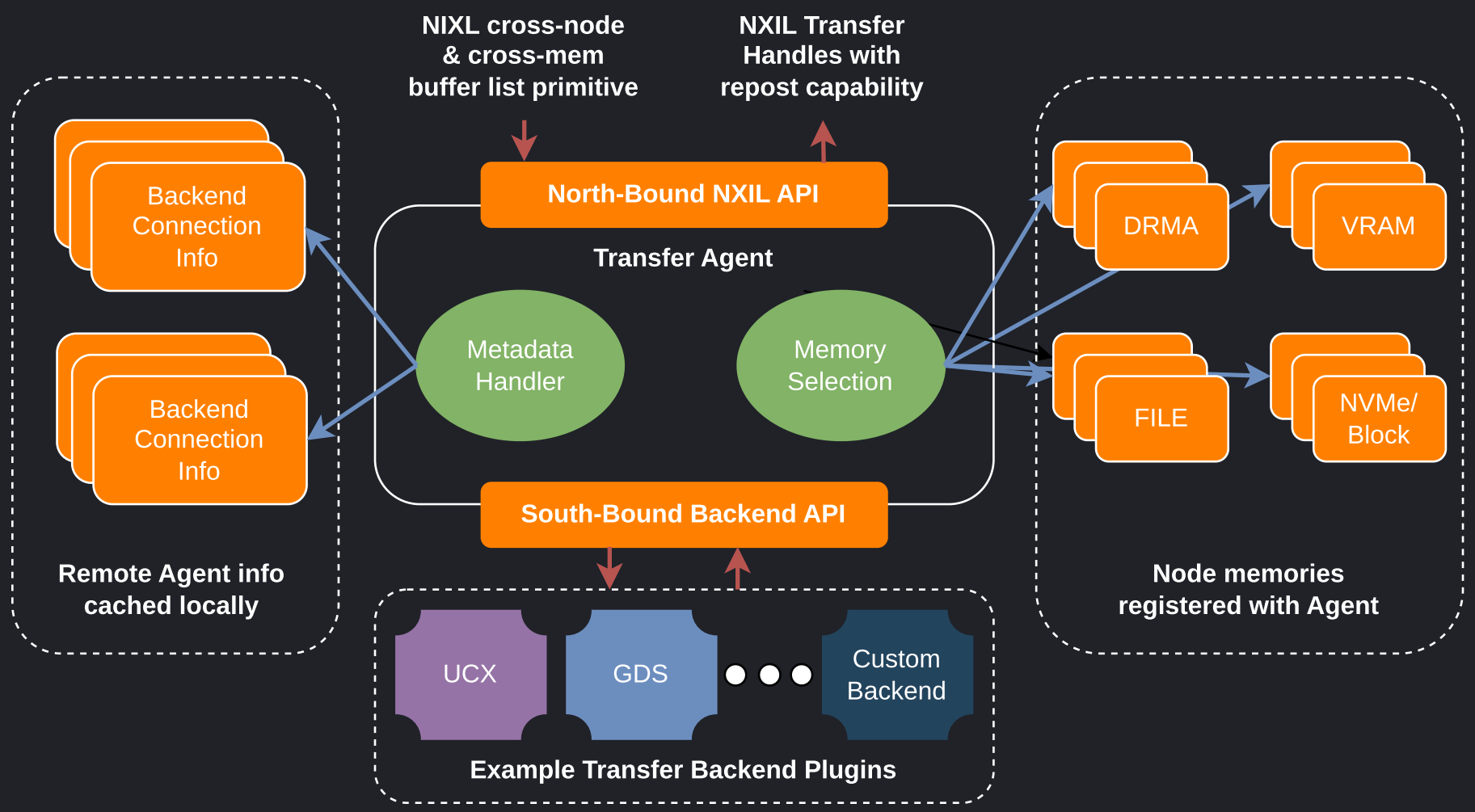
- SharedStorageConnector: サンプル・デバッグ実装。ローカルディスクに書き出す。
- MultiConnector: 復数のV1 Connectorを束ねるメタコネクタのような実装
- P2pNcclConnector: NCCLと統合するコネクタ
- NixlConnectorV1: NIXLと統合するコネクタ
- LMCacheConnectorV1: LMCacheと統合するコネクタ

V0時点では実装があったMoonCakeStore向けコネクタがV1をまだ実装していないように見えるが、LMCacheのレイヤーでは既に統合されている

# NIXL (NVIDIA Inference Xfer Library)

- NVIDIA製のXfer Library
  - P2P通信の高速化を目的としたライブラリ
  - AI Inference FrameworkにおけるTransportとして利用する想定
- NVIDIA Dynamoで利用されている
- データ転送におけるストレージソリューションと最適な転送方式を抽象化する
  - つまり、P2PでKV Cacheを転送を最適に実行するためのライブラリ
  - モジューラプラグインアーキテクチャを採用しており独自に拡張可能

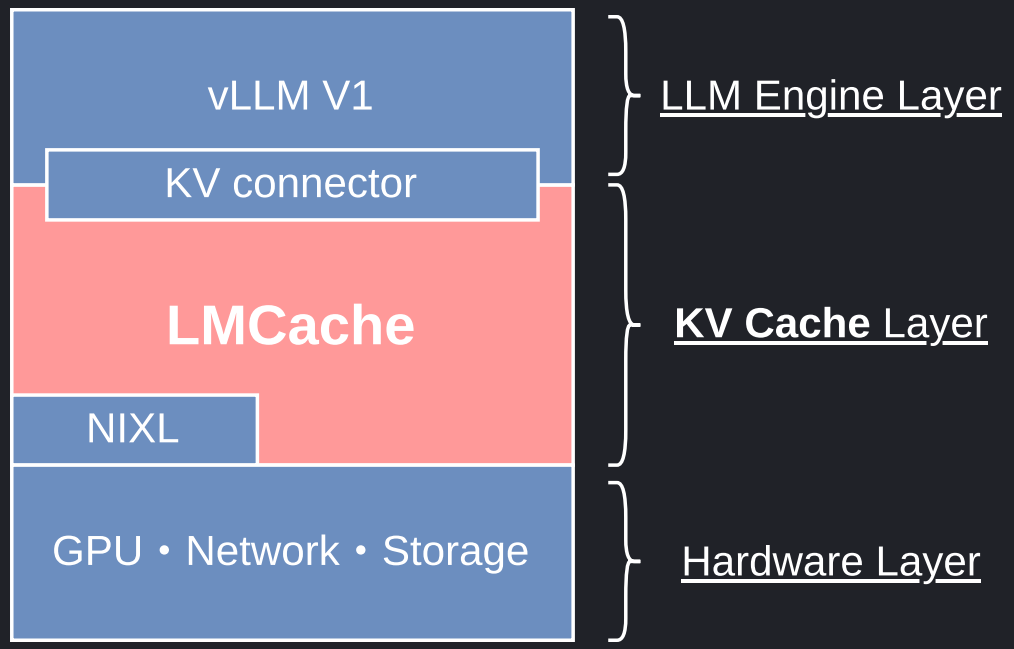
# NXIL Overview




ref: https://github.com/ai-dynamo/nixl/blob/main/docs/nixl.md

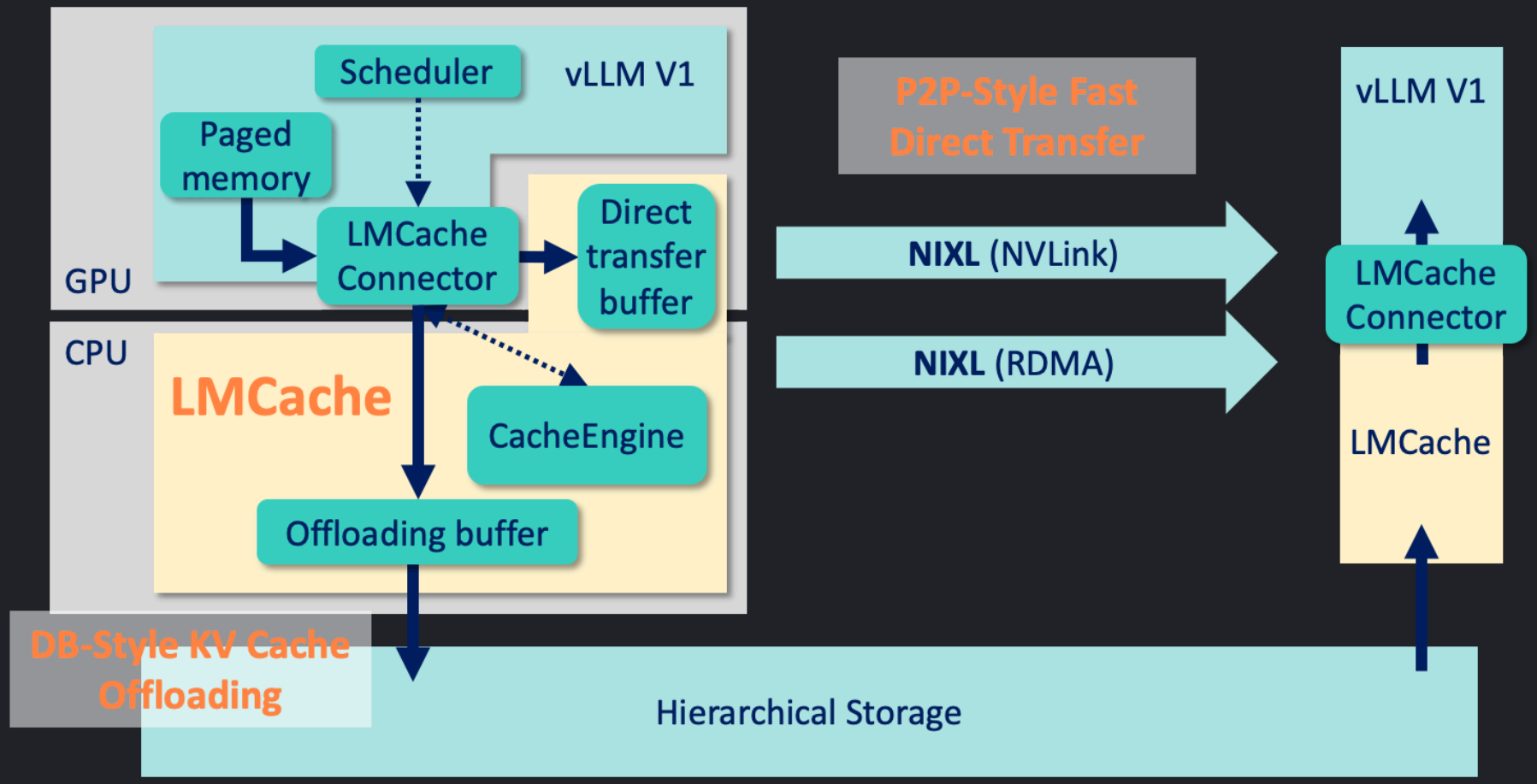
# LMCache

- LMCacheはLLM Serving Engineの拡張機能として、KV-Cacheの管理機能を提供する
- ロングコンテキストのシナリオにおいて、TTFTを削減しスループットを向上させる
- KV-cacheのバックエンドとして様々な場所（storageなど）に保存することができる
- LMCache ConnectorはvLLM公式のUpstreamと統合されているためシームレスに利用可能




ref: Shaping NIXL-based PD Disaggregation in vLLM V1

# LMCache: Two Core KV Cache Functions




ref: Shaping NIXL-based PD Disaggregation in vLLM V1

# Storage: DB-Style Loading and Offloading

- KV Cacheをストレージソリューションに永続化する
- **ユーザセッション間でKV Cacheを共有する**際などに有効に利用できる
- DB-styleの抽象化によって、GPUからCPUメモリやDISKなどにKV-cacheをオフロード
  - EvictionやPrefetch戦略もカスタマイズ可能
- LMCacheのvLLMとの統合はvLLM V1の高度なメモリ機能（prefix caching, chanked prefill, paged memory manager）に合わせて設計されている

# Transport: P2P-Style Direct Transfer

- PD disaggregation向けに、リアルタイムでKV-cacheを別ノードに転送する仕組み
- LMCacheはP2P KV transferをサポートしノードをまたいた再計算をなくしキャッシュを有効に利用する。
  - NIXLをサポートすることで多様はハードウェア、トランスポートに対応
- このNIXLによる高速なKV-cache transferは、多くの場合単一のトークンのAttentionを計算するより高速

# KVCache sharing strategies

LMCacheには現在2種類のKV-cache sharingの方式が存在する。

- **Centralized Sharing**: 中央のキャッシュサーバを利用したKV Cacheの共有
- **P2P Sharing**: Peer-to-PeerでKV Cacheを共有する

以降では、llm-dの議論の際にすこしだけ議論に絡むため、P2P Sharingの方式について軽く深堀りする。

＊ あくまでKV-cache共有によるPrefill段階での計算量削減の文脈。ややこしいが後者はPD Disaggregation時のP2P Transferの話とは別軸の話

# Example: KVCache P2P Sharing: CPU RAM

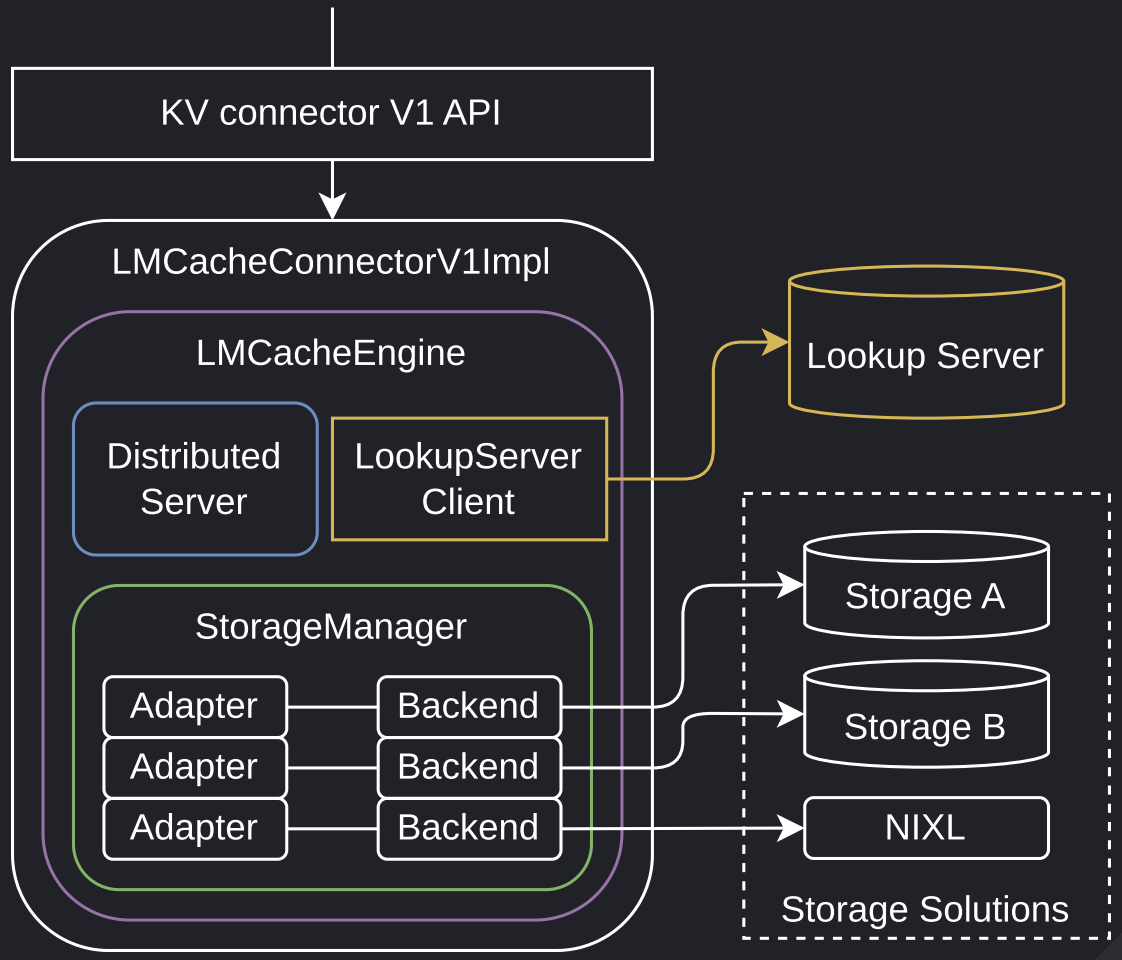
KVCache P2P Sharingを行う場合、有効化設定である eanble_p2p に加え、lookup_url と distributed_url という設定項目が必要になる

| YAML Configuration Name | Environmental Variable | Default |
|---|---|---|
| enable_p2p | LMCACHE_ENABLE_P2P | False |
| lookup_url | LMCACHE_LOOKUP_URL | "" |
| distributed_url | LMCACHE_DISTRIBUTED_URL | "" |

ref: https://docs.lmcache.ai/api_reference/configurations.html#peer-to-peer-sharing-configurations

# LMCacheEngine

- LMCacheEngineにコアロジックの実装がある
  - 様々なStorageBackendをインターフェイス越しに操作
  - LMCacheがサポートするストレージソリューションはこのインターフェイスを実装する形で統合される
- 前述の通りP2P Sharingでは**LookupServer**や**DistributredServer**という概念が登場するが、これは何なのだろう？



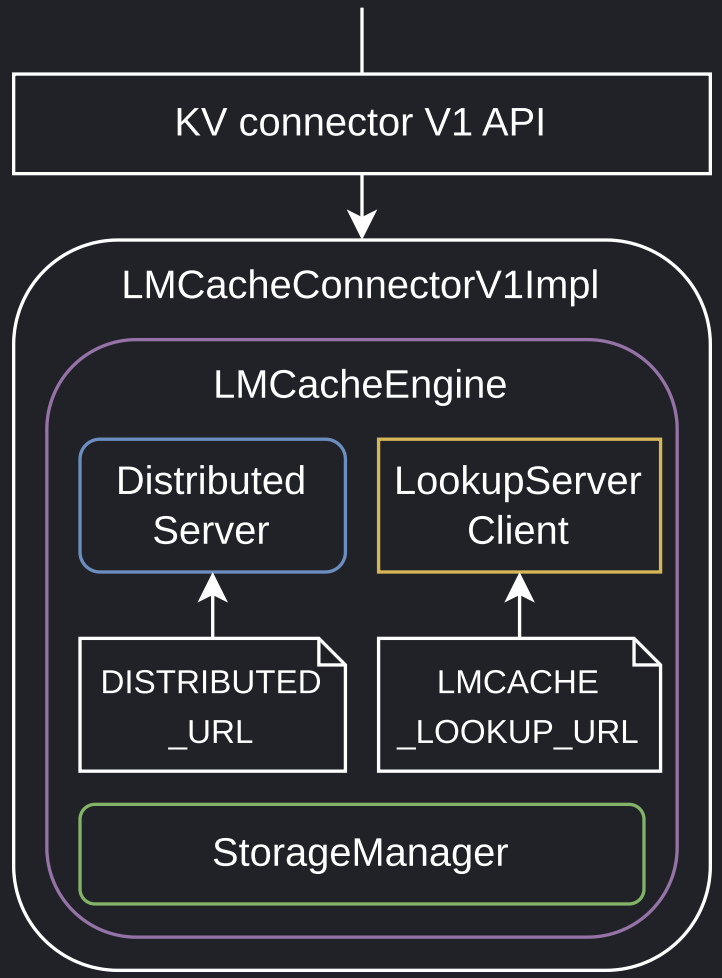
# LookupServer and DistributedServer

**LookupServer**

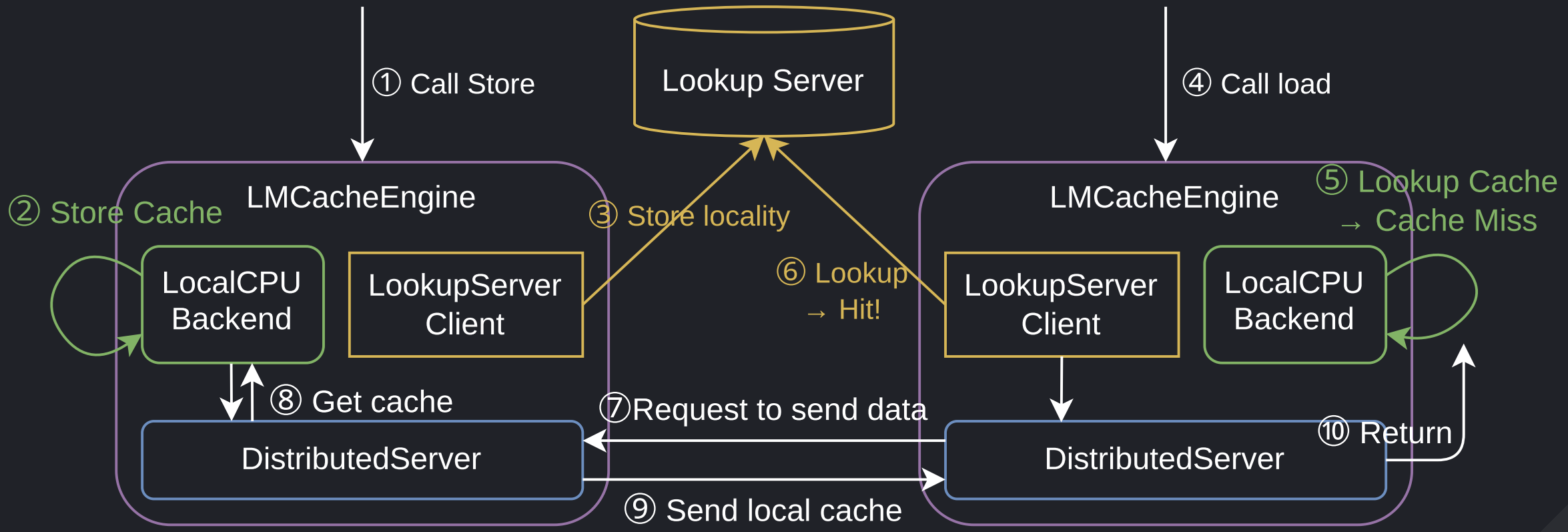
- KV-cacheのローカリティ（host:port）が書き込まれている（外部）Redis Serverのこと
- `LMCACHE_LOOKUP_URL` によってLMCacheEngine内部のRedisクライアントが接続、リクエストを発行する

**DistributedServer**

- KV-cacheをP2Pで転送しあうときの送受信サーバ（LMCacheEngine内の別スレッド）
- `LMCACHE_DISTRIBUTED_URL` によって指定したホスト、ポートで通信を待ち受ける



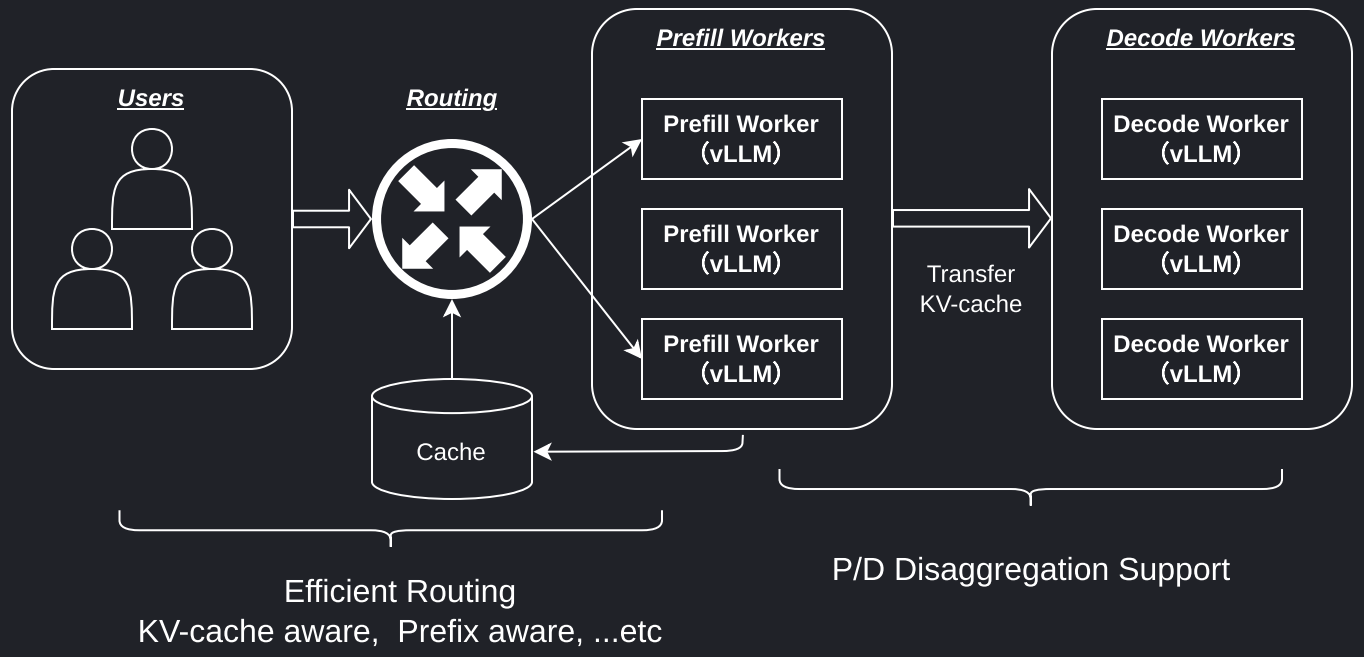
# How P2P KV-cache sharing works?



# LLM Inference Framework: llm-d

# LLM inference considertaion

LLM Inferenceの最適化技術を最大限活用できる最適なルーティング、P/D Disaggregationを効率的に実行できるインフラの構成などを考える必要がある。



# llm-d

- llm-dはvLLMとKubernetesをベースとした分散推論基盤ソフトウェア
- 大規模で最適なGenAI推論基盤の提供を目指すオープンソースプロジェクト
- RedHat, CoreWeave, Google Cloud, IBM Research, NVIDIAがローンチメンバー
- AMD, Cisco, Hugging Face, Intel, Lambda, Mistral AIがパートナー

llm-d

ref: LLM-D.AI

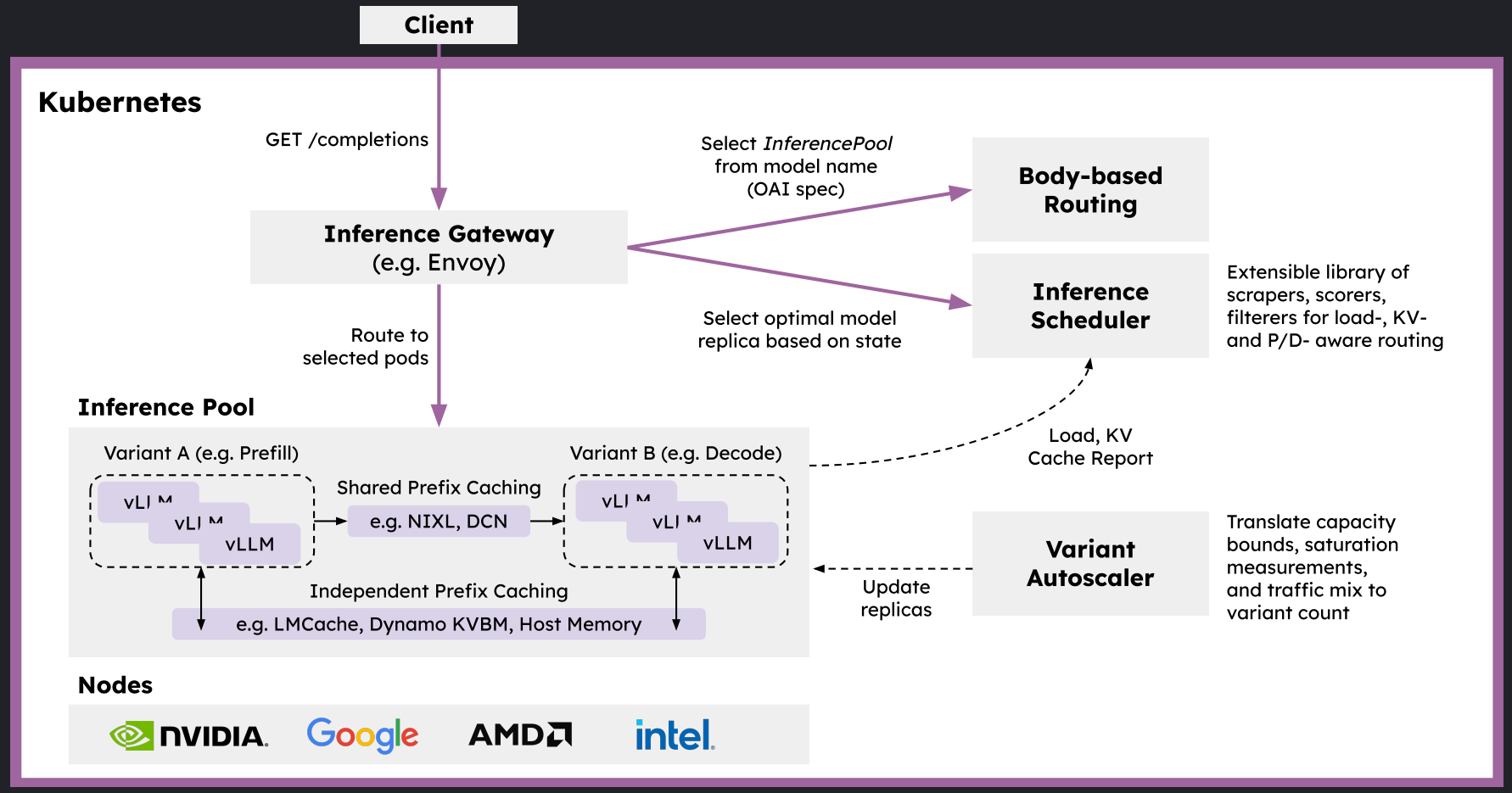
# llm-d

llm-dはkubernetes環境に以下の機能を提供する

- ModelServiceによる一貫したLLM Modelのデプロイ
  - vLLMのPD Disaggregationを補完するデプロイ機構の提供
- Gateway API Inference Extensionをベースとしたリクエスト推論向けルーティング機能
- vLLM, LMCacheなどの最適化手法を最大限活かせる環境
- 柔軟なAutoScaling技術 (WIP)

ref: Overview of llm-d architecture

# llm-d architecture



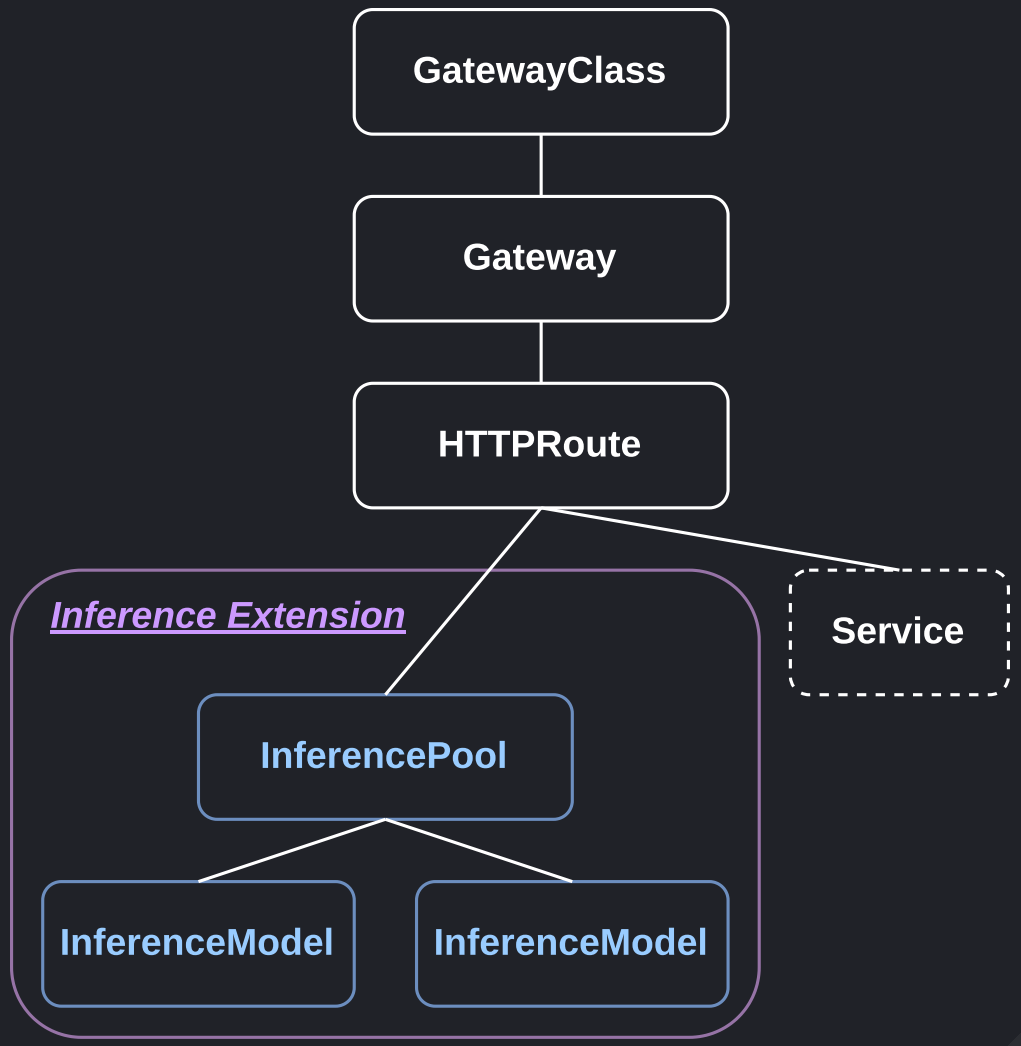

ref: https://github.com/llm-d/llm-d

# Inference Gateway (IGW)

- Gateway API Inference Extension（GIE）をベースにしたルーティング機能の拡張
- LLMの推論リクエストに特化した最適なルーティング機能を提供する
  - KV-cache aware routing
  - Prefix aware routing
  - Session affinity routing
  - Load aware routing
  - Model metadata routing
- ルーティング機構はPluggableであり、機能を拡張することができる。
  - 現状は環境変数で切り替えられる

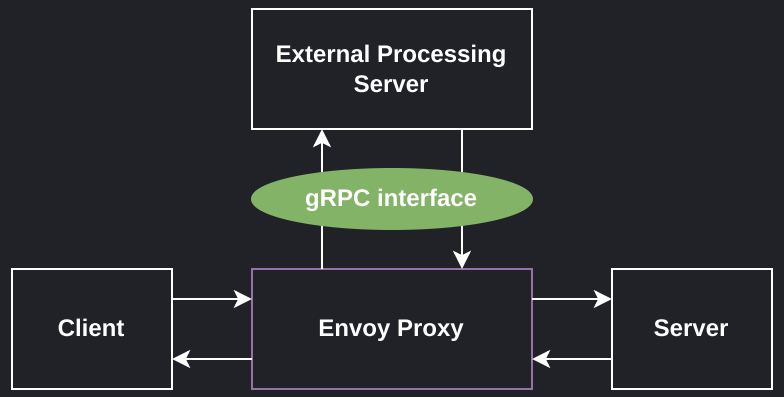
# Gateway API Inference Extension (GIE)

- k8s上にGenerative Modelを最適にセルフホストするための公式プロジェクト
- Gateway APIのHTTPRouteのextensionRefフィールドを利用して機能を拡張する
- 推論リクエストに対して、コスト面・性能面から最適なルーティング・ロードバランス機能を提供
- 詳細は公式資料のIntroductionを参照



# GIE and Envoy External Processing

- GIEの実装方式としてEnvoy External Processing(ext-proc)が利用されている
- ext-procはEnvoy外部のと連携しFilter Chainの拡張を実現することができる機能
- HTTPのRequest/ResponseのHeaderやBody, Trailerの読み込みや修正が可能
- gRPCインターフェイスを実装する外部サービス（external processor）と連携可能

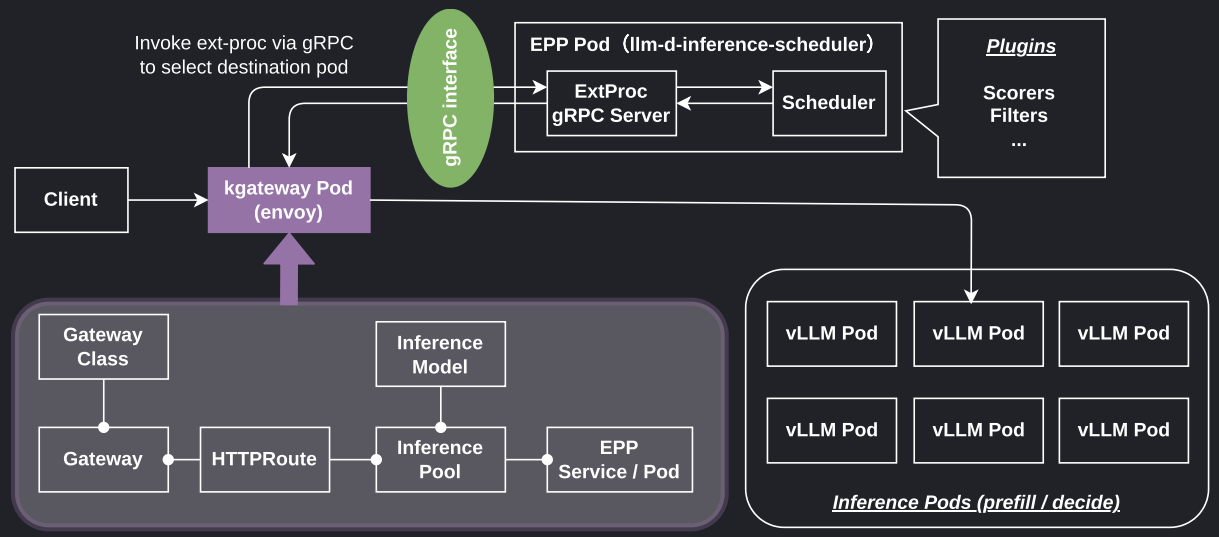



ref: External Processing Filter

# GIE implementation @kgateway

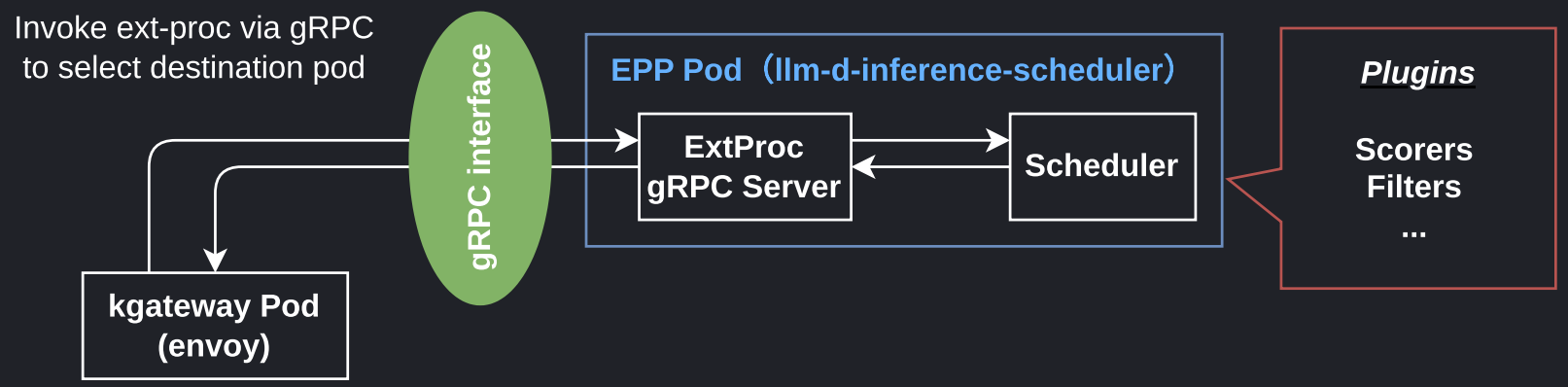
**Ex. kgateway（envoyベース）**

- GIEのリソース（HTTPRoute, InferencePool, InferenceModel）を作成すると、ext-procのため設定を生成し読み込む
- ext-procのリクエスト先は、InferencePoolで定義しているバックエンドのリソース
- 以降、HTTPRouteで指定したInferencePool向けのルールにマッチするリクエストを受けると、ext-procへのgRPCリクエストを発行するような挙動を行う



# Endpoint Picker (EPP) & llm-d-inference-scheduler

- GIEにおいて Endpoint Picker (EPP) がスケジューリングのコア機能を担う
- llm-dではこの機能は llm-d-inference-scheduler に実装される
- llm-d-inference-schedulerはGIE本家の実装をベースに、GIEの拡張可能性を利用した独自のscheduling機能の実装（Plugins）を行っている
- ext-procとしてgRPCを受け、scheduling機能によって最適なエンドポイントを選択する

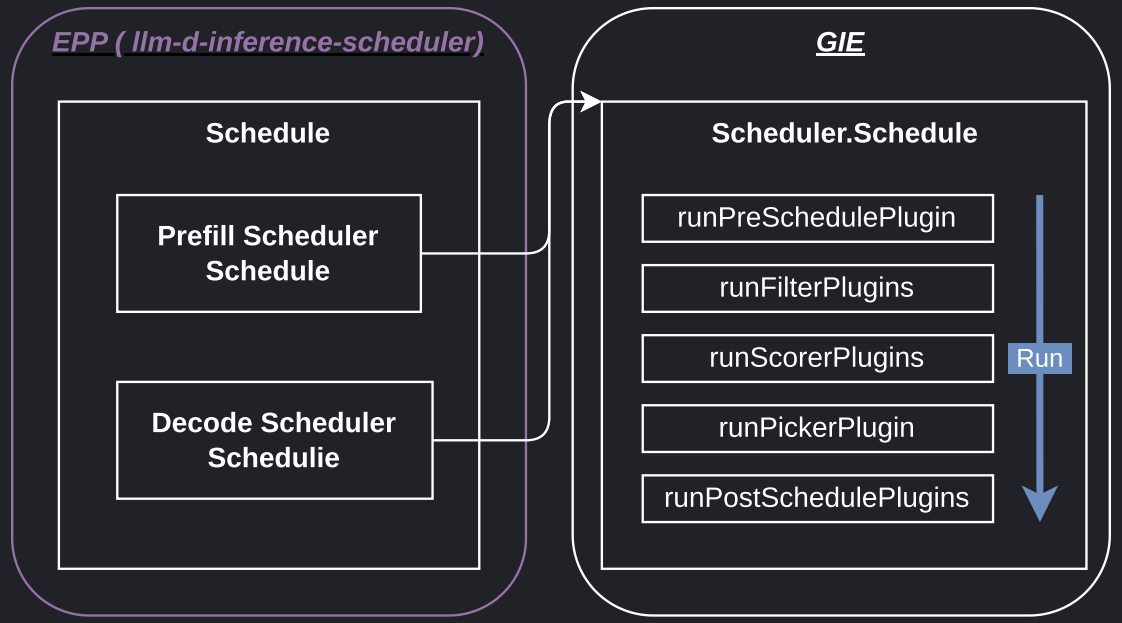


# Scheduling processes of EPP

EPPは、GIEのリファレンス実装のSchedule処理をそのまま呼び出し、以下のような処理を実行する。

- PreSchedulePluginの実行
- FilterPluginの実行
- ScorerPluginの実行
- Pickerの実行
- PostSchedulePluginの実行

GIEではそれぞれのPluginなどが満たすべきインターフェイスが定義されており、それを満たすような実装を行うことで処理の追加や差し替えが可能。




＊EPPの実装的に処理の分岐等はあるが、必ずPrefill・DecodeのどちらかのSchedulerでGIEのSchedule処理は呼び出される形にはなる

# llm-d defined scorer

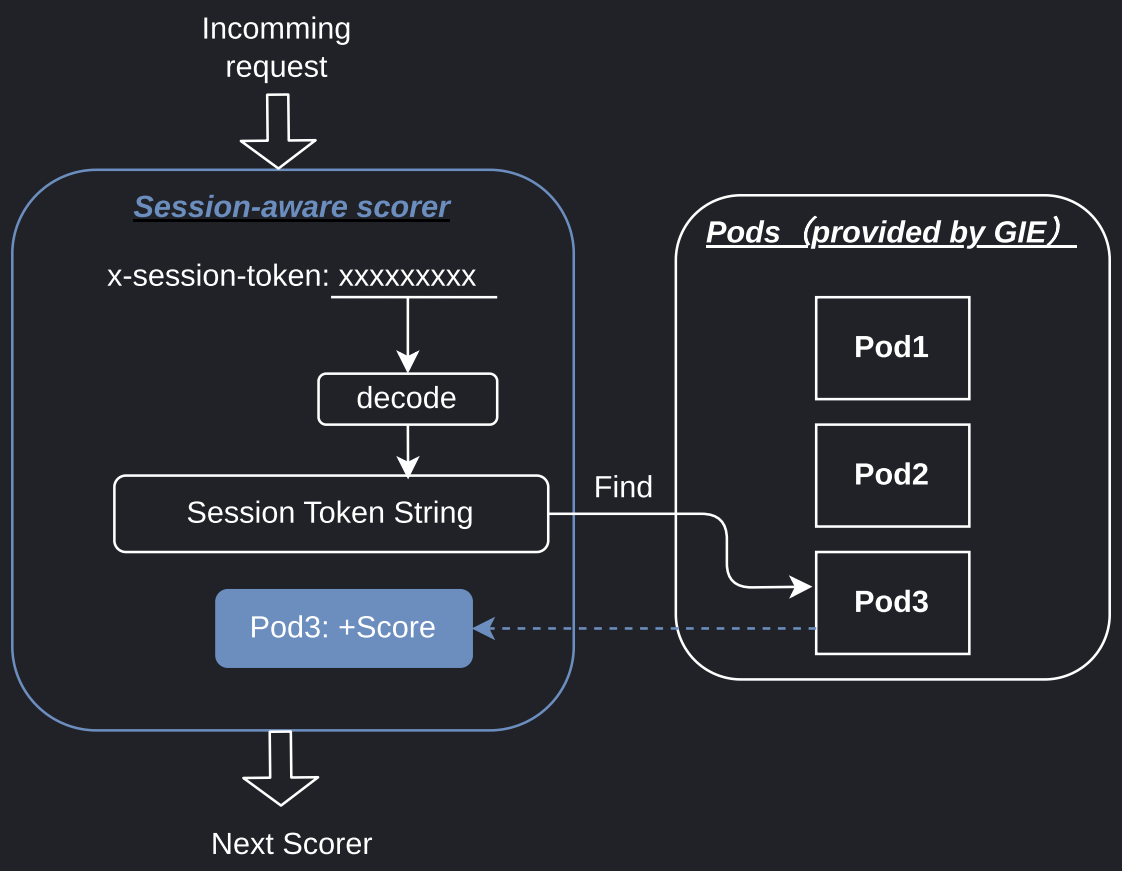
llm-dではGIEのSchedling処理の中で拡張可能なFilterやScorerとして独自の機能を持つものを定義している。例えば、llm-dのScorerには以下が存在する。

| Scorer | Description |
|---|---|
| Session-aware | Session IDに基づいて、直前にそのセッションを処理したPodを優先 |
| Load-aware | Podのリアルタイムな負荷指標を元に過負荷なPodを避ける |
| Prefix-aware | Promptのprefixが過去に処理されているPodを優先 |
| KVCache-aware | KVCacheの再利用が見込めるPodを優先 |

これらのScorer有効・無効などは環境変数から設定可能な実装になっている。

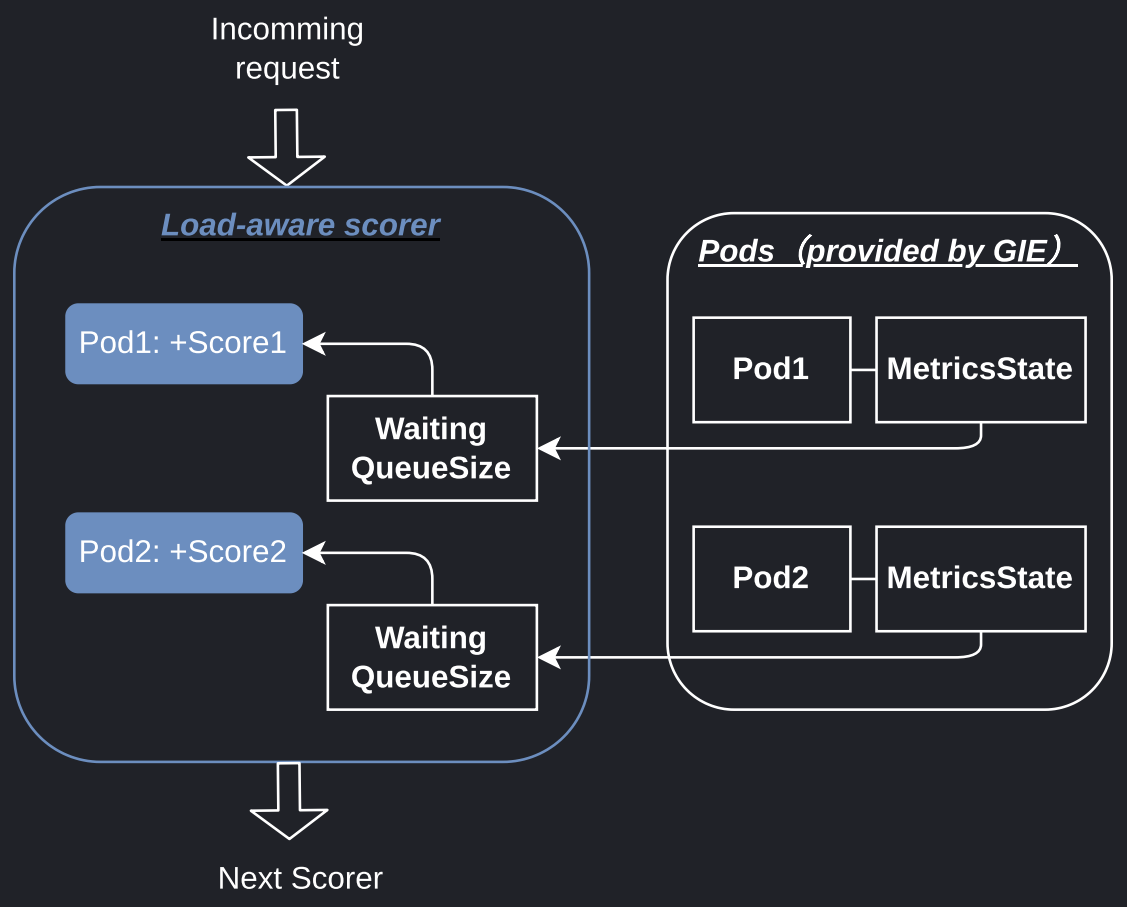
# Session-aware scorer

- リクエストに含まれる、「x-session-token」の情報を取得し、Podの情報が書き込まれている場合は対象のPodのスコアを加算する



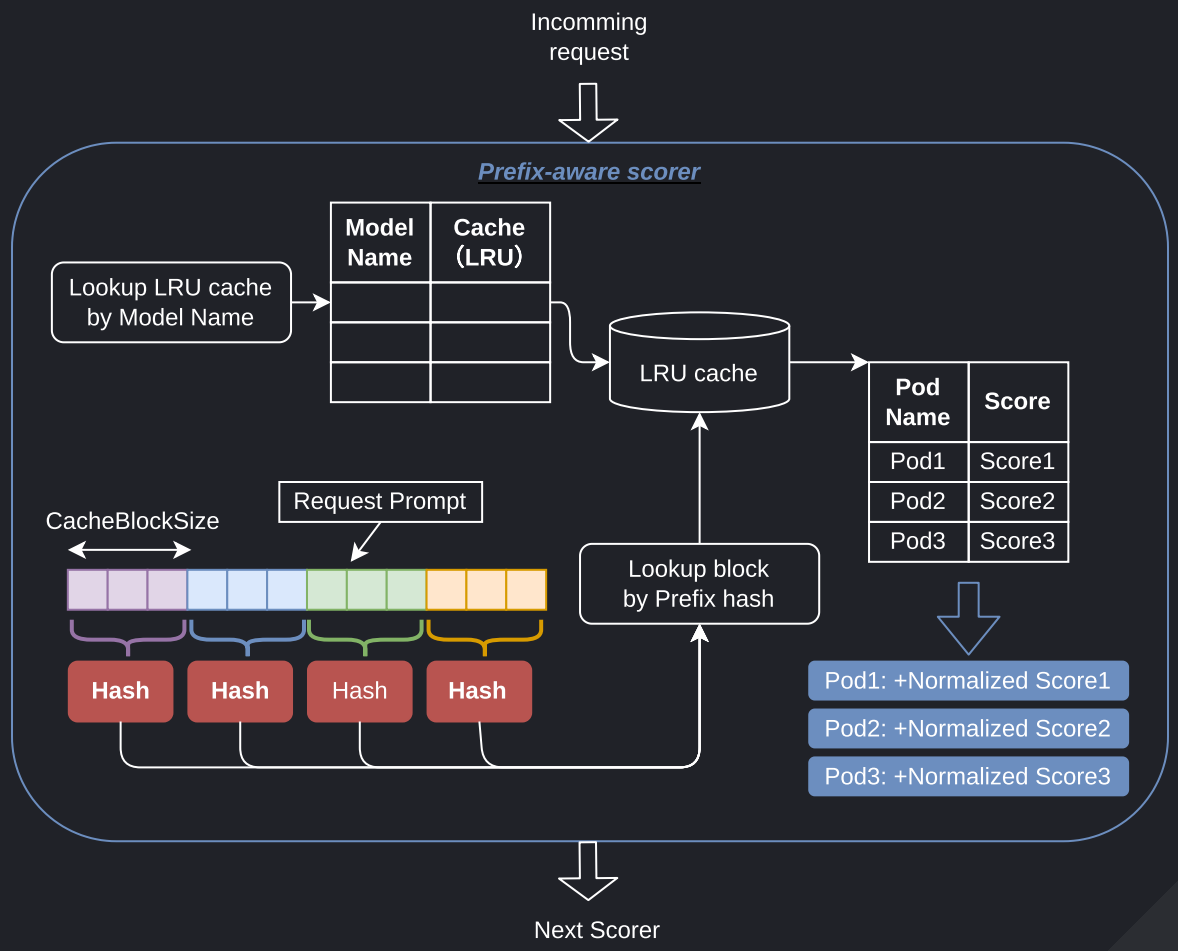
# Load-aware scorer

- Podの負荷状況に合わせて、負荷の小さいPodを優先するようスコアリング
- PodのメトリクスはGIEの実装の中で定期的にPodのメトリクスを収集・保存する仕組みがあり、その情報をスコアリング時に利用している
- 現状の実装ではWaitingQueueの情報のみを利用している



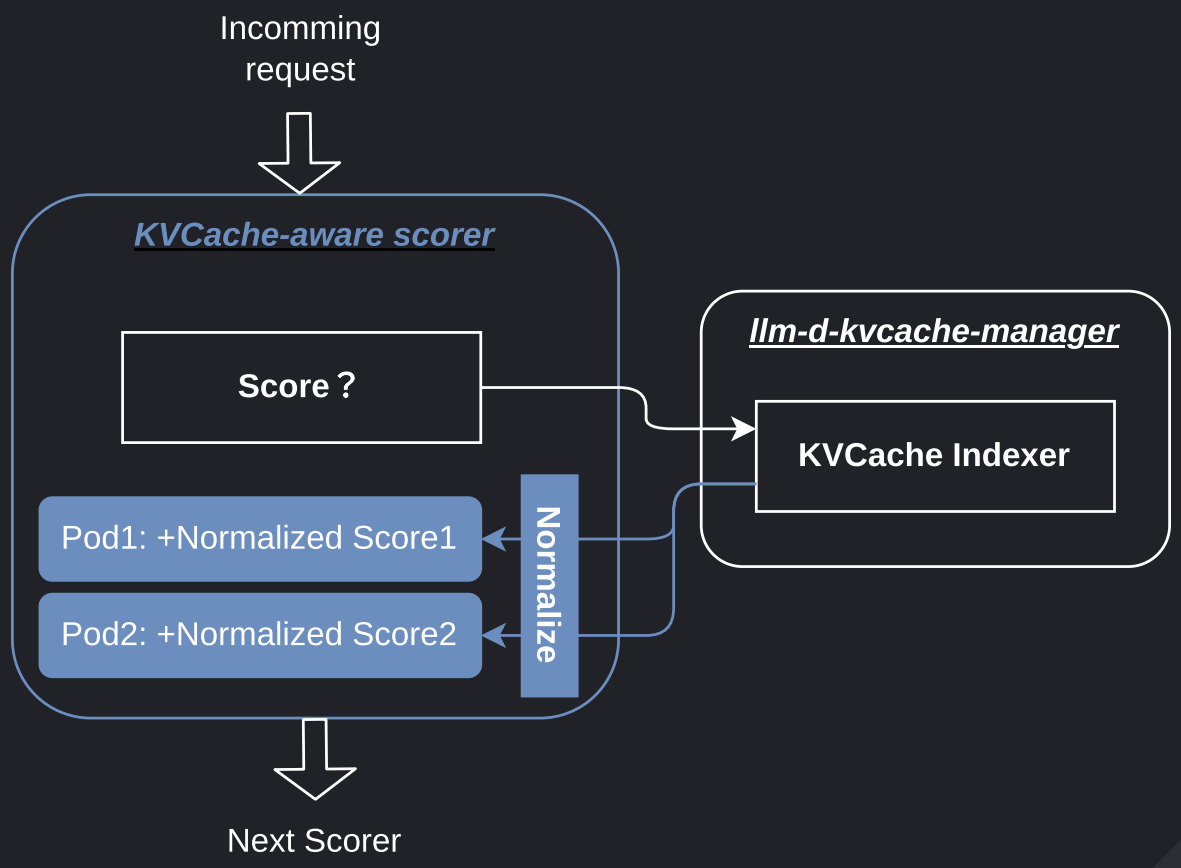
# Prefix-aware scorer

- リクエストのPromptのPrefixが過去にどこかのPodで計算、キャッシュされている場合、それを検出してスコアを上乗せする。
- インメモリにLRUキャッシュを構築し、リクエストを受けるとこれをLookupしてPodに対してスコアリング



# KVCache-aware scorer

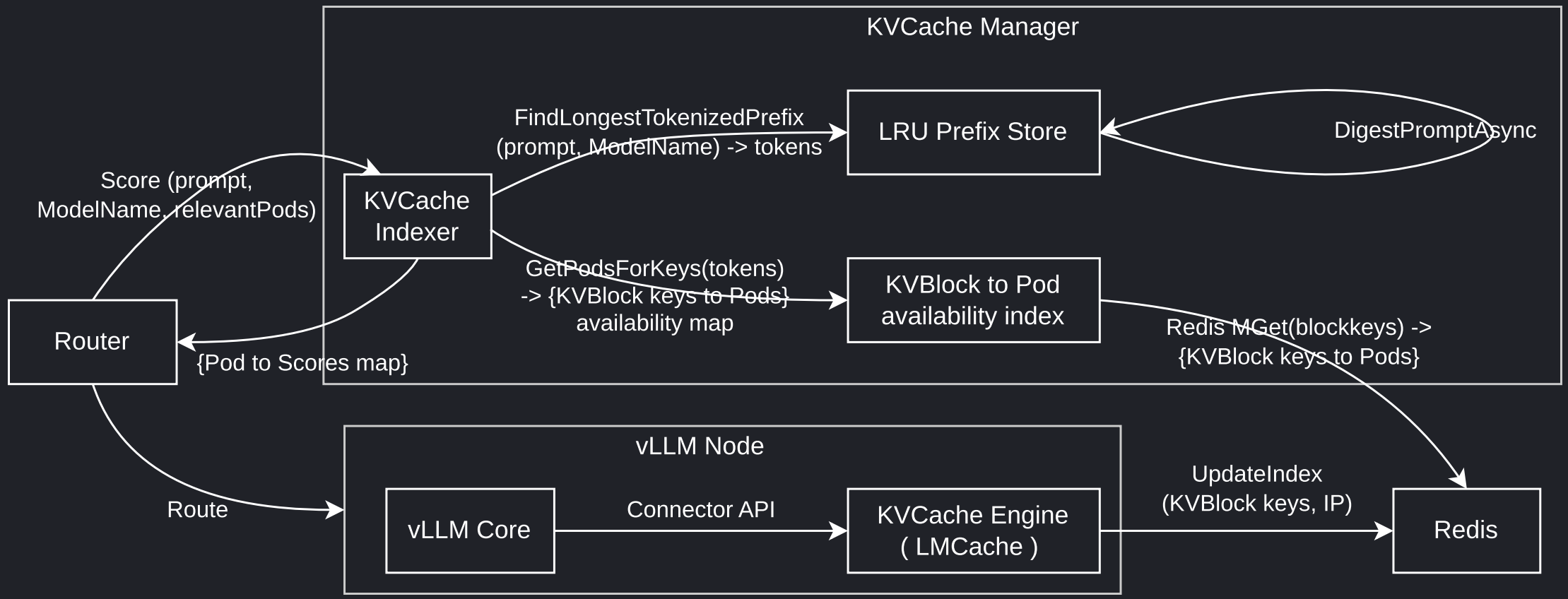
- KVCacheが再利用できるように最適なルーティング先のスコアを上乗せする
- llm-d-kv-cache-managerにリファレンス実装があり、KVCache-aware scorerは基本その実装を利用する形で実装されている。



# llm-d-kv-cache-manager

- KVCache-awareなルーティングを実現するための管理機構
  - リクエストに対し、どのPodにルーティングすると最もながいPrefixでKVCacheにヒットするか、を最適化するようなルーティングを実現したい
- 現在の実装ではLMCacheを概ね前提にしていそう
  - 現時点ではさらに、vLLMのConnector APIを介した、KVCache Engineにオフロードされるテンソル情報のみしか利用できない
- KVCacheIndexerという実装があり、これがKVCacheを意識したスケジューリングを行えるように設計された、プラグイン可能なモジュールになっている。
- Scorerのリファレンス実装も存在する。KVCacheIndexerを利用し、KVCacheを最適に再利用できるPodのスコアを加算するようなスコアリングの実装が存在する。

# llm-d-kv-cache-manager overview




ref: https://github.com/llm-d/llm-d-kv-cache-manager

# llm-d-kv-cache-manager implementation

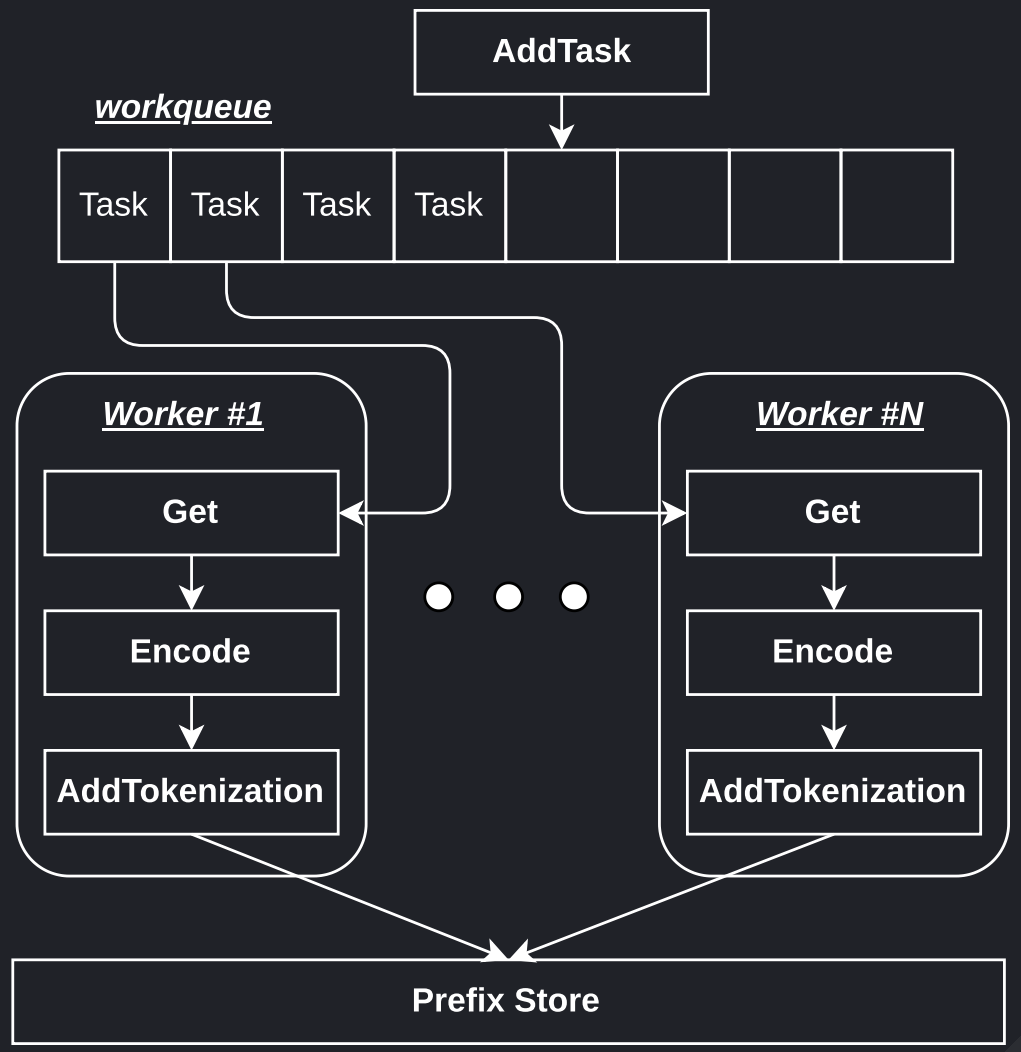
KVCacheIndexerはいくつかのコンポーネントで成り立つ

- tokenization.Pool: インメモリのQueue構造＋それを処理するWorkersの実装
- prefixstore.Indexer (&LRUTokenStore): インメモリキャッシュへのIndexer
- kv-cache.TokenProcessor: Token処理を行うプロセッサ
- kv-cache.Indexer: （外部のRedisに対するIndexer）
- kv-cache.KVBlockScorer: 戦略に従ってスコアを計算する実装

llm-d-inference-schedulerで利用されるスコアリング機能を元に少し深堀りしていく

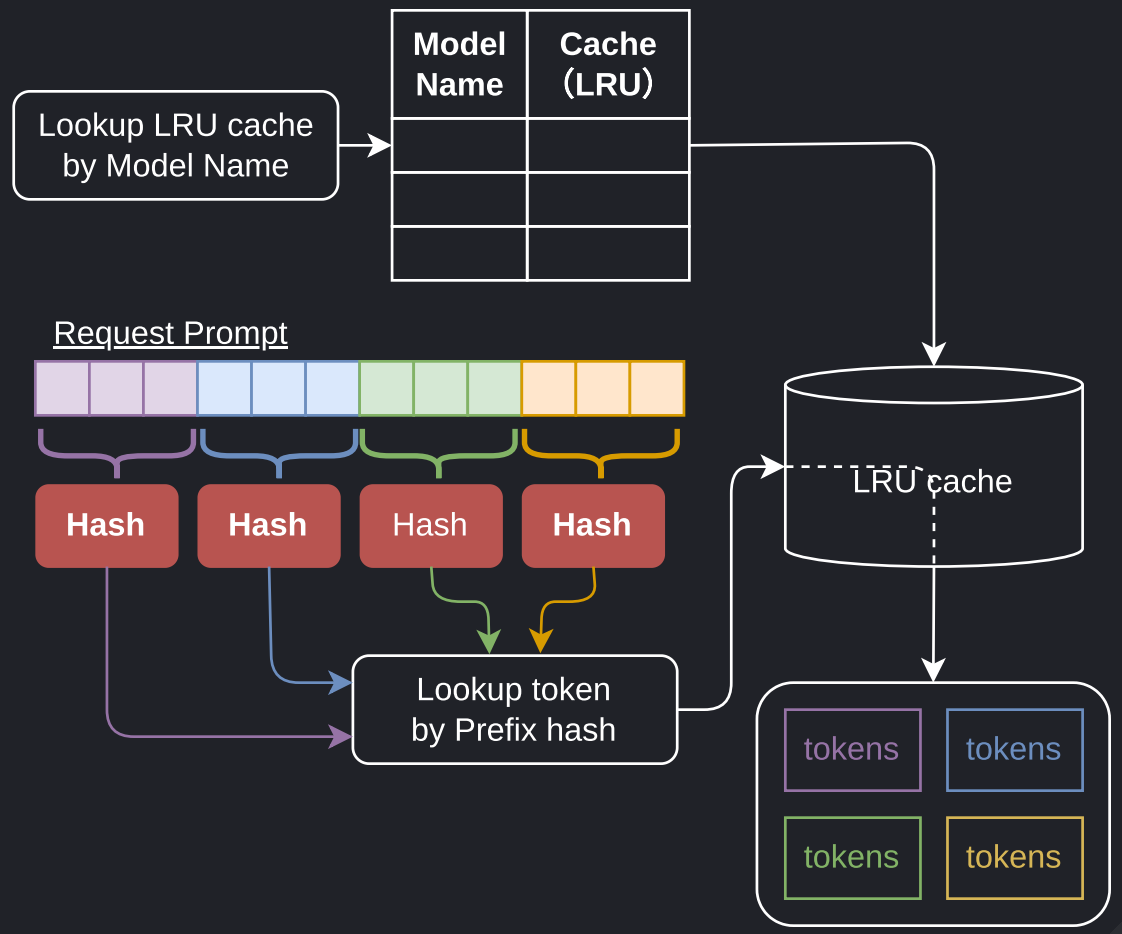
# tokenization.Pool

- リクエストを一時的に保存するQueue構造 + Queueを処理してインメモリLRUキャッシュ（prefixstore.Indexer内部の構造）にtokenizeして書き込むWorker
- QueueへのTaskの追加は別のプロセス・処理から追加される（llm-d-inference-schedulerの場合はScoring処理の冒頭で追加している）



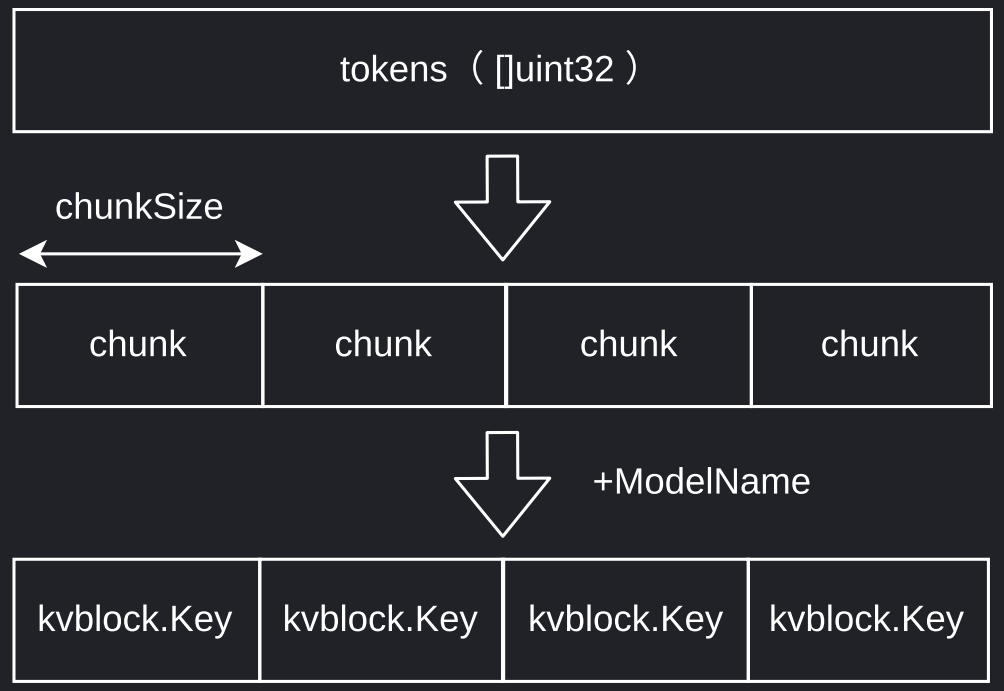
# prefixstore.Indexer

- インメモリLRUキャッシュ構造、及びLRUキャッシュを索引するインデクサ
- LRUキャッシュに書き込まれたTokenをリクエストのモデル名、Promptから索引できる
- Scoringの文脈では、PromptのPrefixの最長マッチして、取得したTokensをつなげたものを返却する実装がある



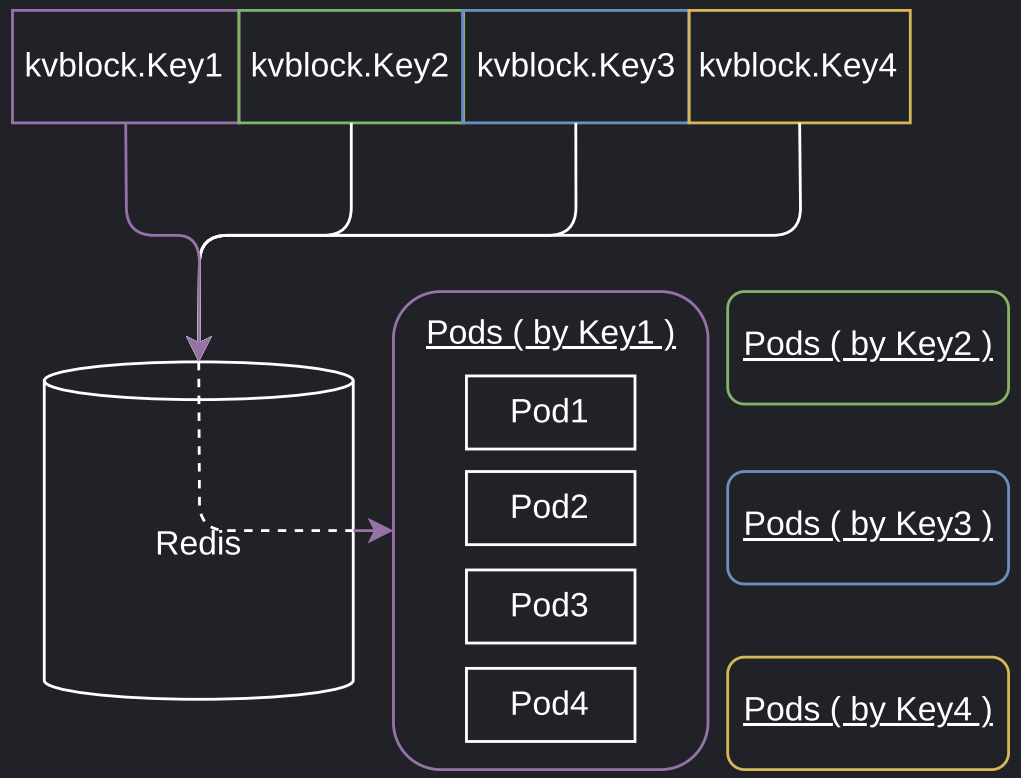
# kv-cache.TokenProcessor

- Tokenから（外部）Redisの索引に利用するKey（Block Key）を作成するプロセッサ



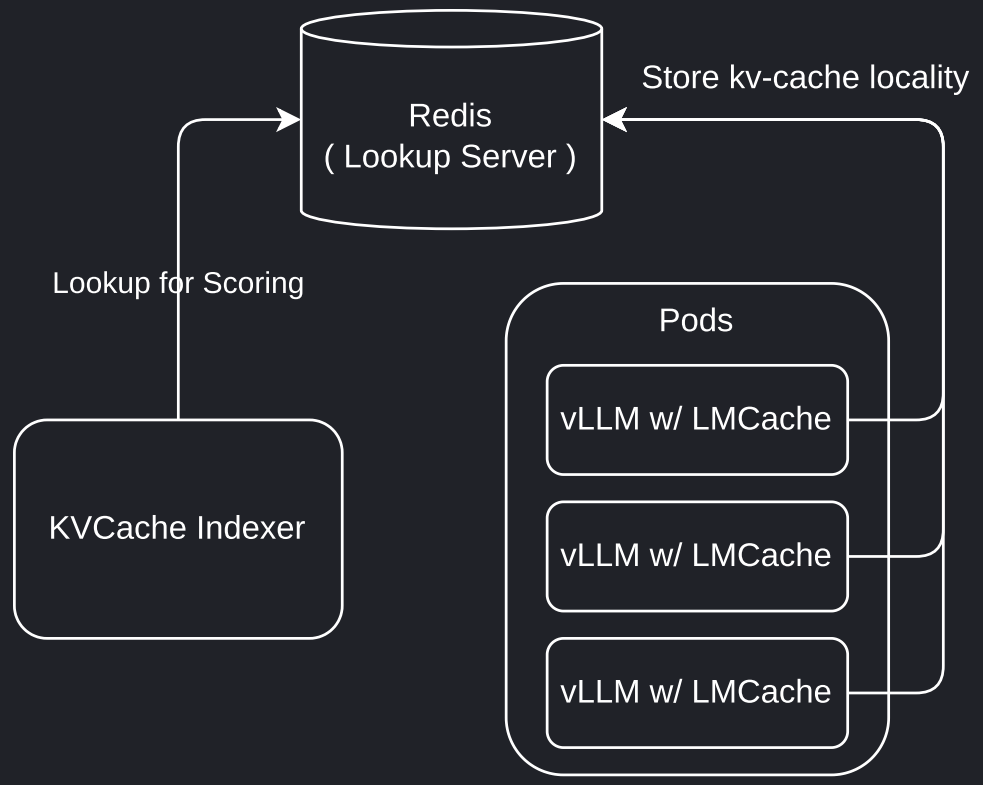
# kv-cache.Indexer

- Block Keyを元にRedisを索引するインデクサ。Podsのリストが取得される。
- 今回の場合はRedisにはvLLMから....なので「対象のPrefixを処理したPodのリストを取得する」という形になる



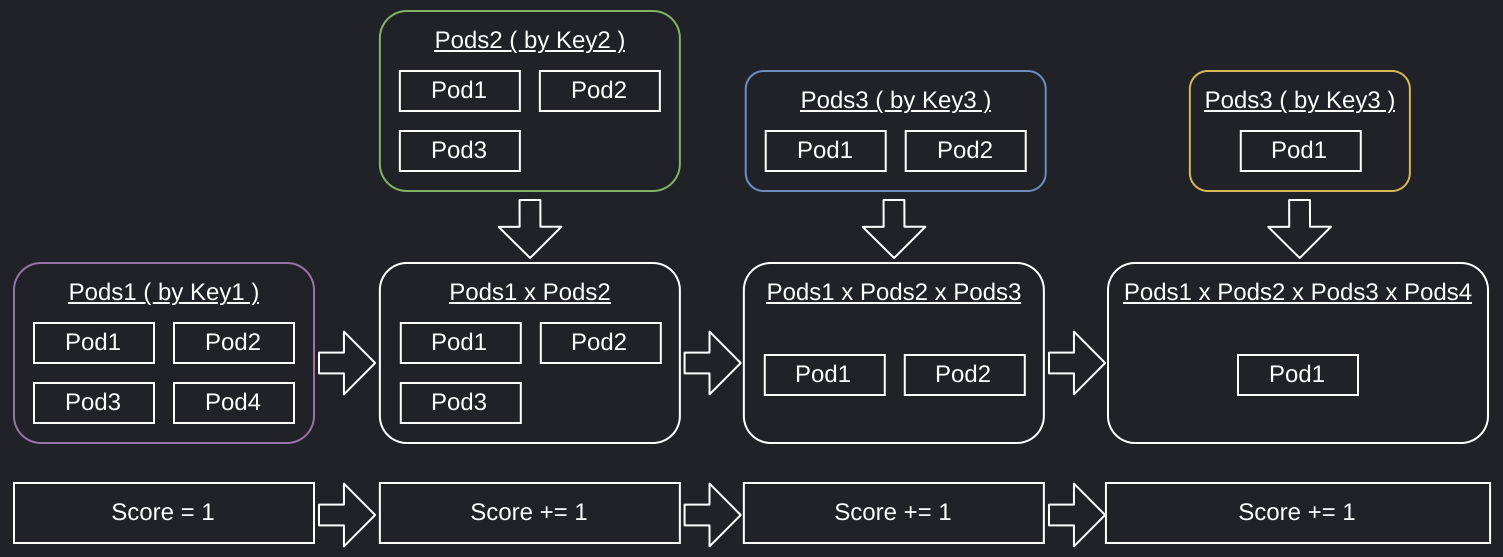
# What is this Redis? ... LMCache's LookupServer!

- 「RedisからPodのリストを取得」という話をしたが、このRedisがLMCacheで説明したLookup Serverに該当する。
- Lookup ServerにはKV-cacheのLocality（Host:Port）が記述されているため、ここからHost（= Pod）を取り出せる
- ただし、この実装は現在アップストリームでは非推奨としてより良い形にするべく絶賛議論が進んでいます



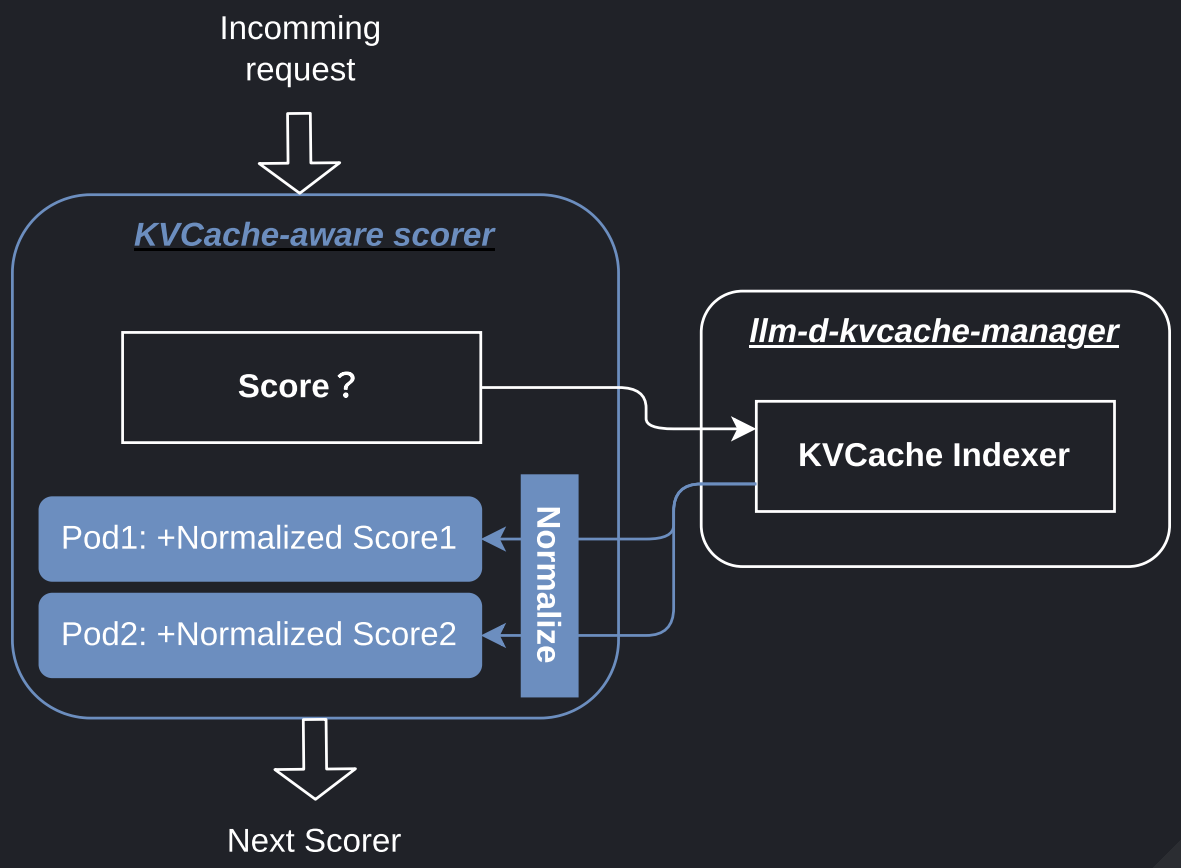
# kv-cache.KVBlockScorer

- kv-cache.Indexerから得られる結果を元に、特定の戦略に従ってスコアリングする
- LongestPrefixScorer（デフォルト）では、Prefixにより長くマッチするKVCacheを保有するPodが一番点が高くなるようにスコアリング



# Revisit: KVCache-aware scorer

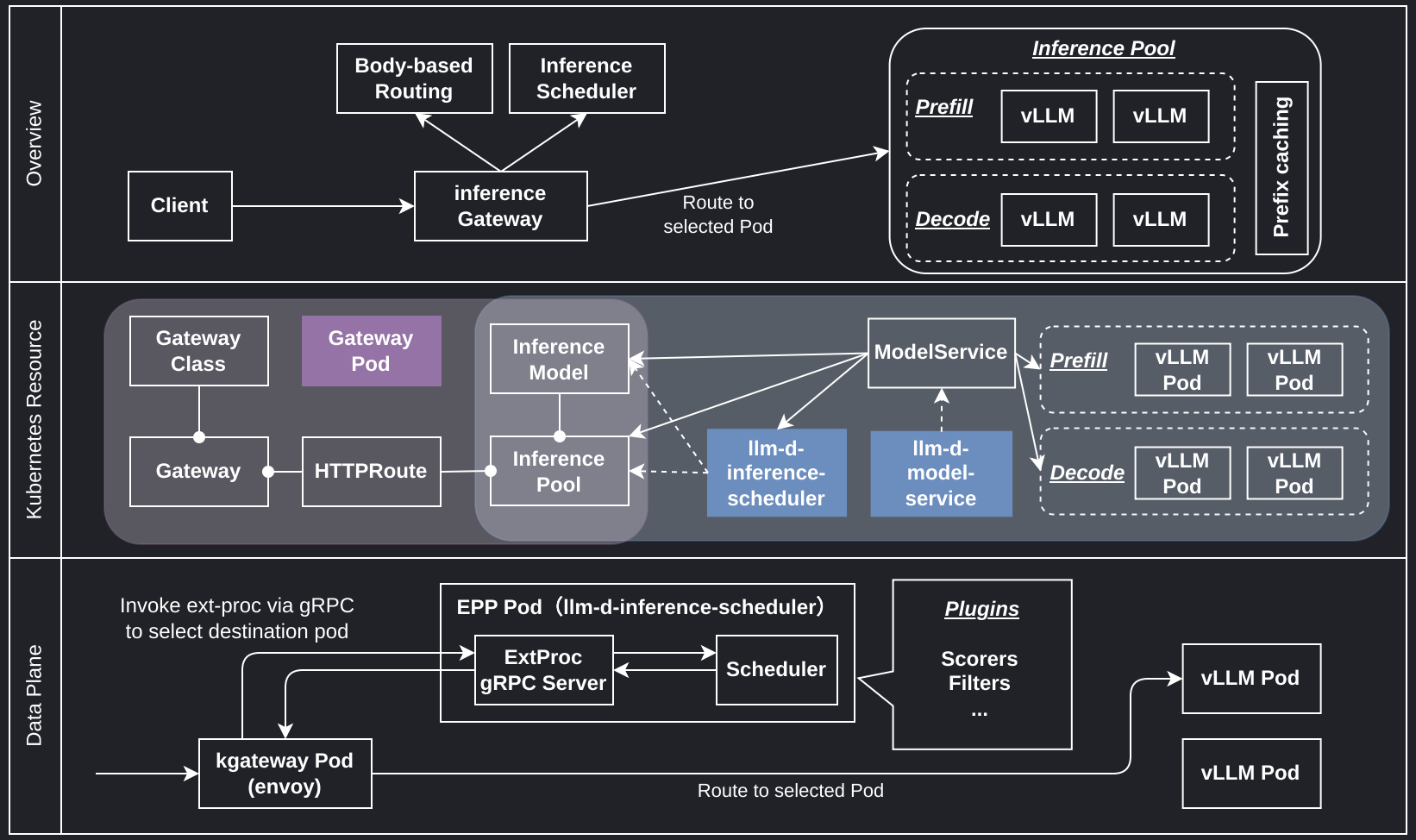
- llm-d-inference-schedulerでは、llm-d-kv-cache-managerで実装されているデフォルトのKVCacheIndexを初期化して利用
- tokenization.Poolはメインスレッドとは別のgoroutineで起動。
- KVCache-aware scorerはこのkv-cache.KVBlockScorerの実装を概ね流用し、スコアリング結果を正規化だけしている。



# llm-d-model-service

- LLM推論 on k8sは、かなり多くのResourceの連携によって実現することになる
- 単一のベースモデルに対して、これらのリソースをうまくコントロールする仕組みを提供
  - ModelService CRにprefil/decode/epp/LLM model情報などを記載
  - ModelService ControllerのReconcile処理の中で様々なリソースを管理
    - Create/Update Prefil/Decode deployment & configmap
    - Create/Update EPP service & deployment
    - Create/Update InferencePool/Inference Model
    - ...etc
  - より詳細なリソース間の関係はREADMEのHow It Worksを参照されたい

# llm-d overview, resources and data plane



# LLM推論における性能指標

# LLM推論における性能指標

本資料では、応答性能に関する代表的な指標と、モデルの精度の指標となるものとして代表的な以下のものを取り上げ簡単に概説するに留める

| 何に対する指標か | 代表的な指標 |
|---|---|
| 応答速度に対する性能指標 | TTFT / TPOT (ITL) |
| モデルの精度（Code Generation）に対する指標 | Pass@1 |

詳細の確認や、その他の性能指標については各々確認されたい

# 応答性能の指標: TTFT / TPOT

**TTFT（Time To First Token）**

- リクエスト送信から最初のトークンが返るまでの時間 (= Prefillにかかる時間)

**TPOT(Time Per Output Token), ITL**

- 1トークン生成にかかる時間 (= 1TokenのDecodeにかかる時間 )

ユーザがモデルに対して行ったリクエストに対し、完全な応答を生成するのにかかる時間（待ち時間）は以下のように計算できる。

$$\text{待ち時間} = (TTFT) + (TPOT) \times (\text{生成されるトークン数})$$

# 精度の指標: pass@1

**pass@1**

- モデルのコーディング能力を評価する指標
- モデルが、与えられた問題に対して生成した最初の施行で正しい回答を生成できる確率を示す指標
- モデル間での性能比較や、量子化により性能影響を比較することができる。

Code Generation系ベンチマークツールとしてOpenAIのHumanEvalや、最近ではLiveCodeBenchあたりが有名そう（試してないです）

# まとめ

# まとめ

- LLM推論ではKV-Cacheの取り扱いを始め、計算効率、処理効率を最適化するための技術的な仕組みやルーティングが重要になる
  - vLLMは様々な最適化技術を実装した推論エンジンとして昨今注目を集めている。
  - LMCacheを利用することでvLLMに対してキャッシュレイヤーの機能を統合できる
  - KV-Cacheの転送を最適化するためにNIXLなどのソフトウェア、それに準じたHWやストレージなどが要求されてくる可能性がある
- llm-dはk8s環境へこれらのインフラをk8s的に実現する手段の一つ。その他にもNVIDIA DynamoやKServeなど様々なソリューションが存在。
  - 自身のインフラにとって最適な選択をするための審美眼が必要。
  - 正しい判断をするにはLLM等の技術に関する正しい知識を持つことが重要になる

# さいごに

本資料では具体的なソフトウェアを取り扱いながら、現在の分散推論基盤に求められる機能やその実装方式について詳しく説明してきました。
ただし、今回の資料ではカバーしきれていない領域（Attentionの最適化や各種量子化、推論時のモデル並列、テンソル並列など）もあるため、まだまだ我々我々が把握すべきことは多いと思っています。

本資料が、今後分散推論基盤構築や検証に関わる方にすこしでも参考になれば幸いです。

# References

Official docs / Codes

- vLLM official docs / Codes
- NIXL docs
- LMCache official docs / Codes
- llm-d official docs / Codes

Related Technologies

- External Processing
- Extend Your Data Plane Out-of-Process Using Envoy's External Processing (Youtube)

# References

Optimization Techniques

- Efficient Memory Management for Large Language Model Serving with PagedAttention
- Mastering LLM Techniques: Inference Optimization
- How continuous batching enables 23x throughput in LLM inference while reducing p50 latency
- Continuous vs dynamic batching for AI inference
- PrefixCaching - SGLang vs vLLM: Token-Level Radix Tree vs Block Level Hashing

# References

Transformer

- Transformer, this tech behind LLMs
- Attention in transformers, step-by-step
- How might LLMs store facts
- LLaMa explained: KV-Cache, Rotary Positional Embedding, Grouped Query Attention, SwiGLU